

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING



REPORT
PROGRAMMING INTERGRATION PROJECT

**KNOWLEDGE RETRIEVAL AND QA FOR
COMPUTER SCIENCE COURSES UTILIZING
RAG**

MAJOR: COMPUTER SCIENCE

THESIS COMMITTEE : COMPUTER SCIENCE

SUPERVISOR(s) : NGUYỄN HỨA PHÙNG, Ph.D.

—o0o—

STUDENT : NGUYỄN HỮU CẦU - 2352129

: NGUYỄN QUỐC HIỆU - 2352344

HO CHI MINH CITY, May 2024

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Problem statement	3
1.3	Thesis structure	5
2	Theoretical Background	7
2.1	Large Language Models	7
2.2	Retrieval-Augmented Generation (RAG)	9
2.2.1	RAG pipeline	9
2.2.2	RAG Components	10
2.2.3	Key Characteristics of RAG Frameworks	11
2.3	BERT-based Embeddings for Semantic Retrieval	12
2.3.1	Why BERT for Embeddings	13
2.3.2	Pooling and Normalization Strategies	13
2.3.3	Similarity Metrics	14
2.3.4	Embedding Dimension and Impact on Recall	15
2.3.5	Lightweight Alternatives: MiniLM	16
2.4	Vector Search	17
2.4.1	Vector-Based Retrieval Mechanism	18
2.4.2	Vector database	19
2.4.3	Role of Vector Search in Retrieval-Augmented Generation (RAG)	20
2.5	Reranking in Information Retrieval	21

2.5.1	Cross-Encoder vs Bi-Encoder Architectures	21
2.5.2	When Reranking is Needed	22
2.5.3	BGE Reranker	22
3	Methods	24
3.1	Requirements Analysis	24
3.1.1	Functional Requirements	24
3.1.2	Non-Functional Requirements	25
3.1.3	Data Requirements	26
3.1.4	Constraints	26
3.2	System Design	27
3.2.1	System Architecture Overview	27
3.2.2	Component Design	29
3.2.2.1	Preprocessing Module	29
3.2.2.2	Embedding & Indexing Module	31
3.2.2.3	Query Pipeline Module	34
3.2.3	API and Frontend Architecture	37
3.2.4	Data Flow	38
3.2.4.1	Indexing Flow	39
3.2.4.2	Query Flow	40
4	Implementation and Testing	42
4.1	Overview	42
4.1.1	Technology Stack	42
4.1.1.1	Core Technologies	42
4.1.2	Project Structure	44
4.1.3	Development Workflow	45
4.1.4	Configuration Management	46
4.2	Implementation	48
4.2.1	Preprocessing Module Implementation	48

4.2.2	Embedding and Indexing Implementation	56
4.2.3	Query Pipeline Implementation	62
4.2.4	API and Frontend Implementation	70
4.3	Testing	74
4.3.1	Testing Strategy	74
4.3.1.1	Unit Testing	74
4.3.1.2	Integration Testing	75
4.3.1.3	End-to-End Testing	75
4.3.2	Test Execution Results	75
4.3.2.1	Test Summary	76
4.3.2.2	Coverage Analysis	78
5	Experiments	80
5.1	Experimental Setup	80
5.1.1	Dataset	81
5.1.2	System Configurations	81
5.1.3	Query Type Categorization	82
5.1.4	Implementation Details	83
5.2	Evaluation Methodology	84
5.2.1	Answer Generation Quality	84
5.2.2	Retrieval Performance	85
5.2.3	Impact of Query Rewriting	85
5.2.4	System Performance	86
5.2.5	Conversation Context Memory	86
5.3	Experimental Results	87
5.3.1	Answer Generation Quality	88
5.3.2	Retrieval Performance	88
5.3.3	Impact of Query Rewriting	88
5.3.3.1	Impact by Query Type	89

5.3.4	System Performance	89
5.3.4.1	Performance Breakdown by Query Type	90
5.3.5	Conversation Context Memory	90
5.3.5.1	Context Utilization Rate	90
5.3.5.2	Context Window Effectiveness	91
5.3.5.3	Follow-up Query Quality	91
6	Conclusion	93
6.1	Summary	93
6.2	Achievement of Objectives	93
6.3	Key Findings	96
6.4	Future Directions	97
	References	98
	Appendix A Source Code and Dataset Availability	101
	Appendix B Task Assignment and Project Timeline	102

List of Figures

Figure 2.1	The evolutionary tree of modern LLMs traces the development of language models in recent years [1]	8
Figure 2.2	Conceptual RAG pipeline adapted from Lewis et al. (2020).	10
Figure 2.3	Examples of semantic relationships captured in embedding space, illustrating gender relations, verb tense transformations, and country–capital analogies.	15
Figure 2.4	Illustration of documents and queries represented in a shared embedding space, where semantic relevance is determined by geometric proximity.	18
Figure 2.5	Bi-Encoder and Cross-Encoder architectures comparison [19]	21
Figure 2.6	BGE embedder and reranker in an RAG pipeline [19]	22
Figure 3.1	Complete system architecture showing two main pipelines : Preprocessing and Embedding & Index, and Query Pipeline → API/FE. The diagram illustrates an abbreviated data flow from document ingestion through indexing to query processing and response generation.	27
Figure 3.2	Preprocessing pipeline showing document conversion, OCR processing, parsing, and chunking stages.	30
Figure 3.3	Embedding and indexing pipeline showing text-to-embedding conversion, L2 normalization, and storage in FAISS index with metadata.	32

Figure 3.4 Query pipeline showing client/Frontend interaction with API Backend, vector search in Index Store, reranking with BGE Reranker, and response generation. 34

List of Tables

Table 4.1	Project directory structure (tabular form)	44
Table 4.2	Test execution results by category	76
Table 4.3	Code coverage by module	78
Table 5.1	Dataset Statistics	81
Table 5.2	Answer Generation Quality on CSE Course Materials . . .	88
Table 5.3	Retrieval Performance Metrics	88
Table 5.4	Impact of Query Rewriting on Answer Quality	89
Table 5.5	Query Rewriting Impact by Query Type	89
Table 5.6	System Performance Benchmarks	89
Table 5.7	Latency by Query Type	90
Table 5.8	Context Utilization Rate	90
Table 5.9	Context Window Effectiveness by Conversation Length . .	91
Table 5.10	Follow-up Query Quality Improvement	91

Abstract

This project aims to retrieve information within the scope of CSE¹ courses by utilizing a Retrieval-Augmented Generation (RAG) system to ensure accurate information retrieval. In the current landscape, while powerful Large Language Models (LLMs) are capable of performing information gathering and question answering tasks effectively, they often encounter issues such as hallucination when controlling input information or catastrophic forgetting due to context length limitations. Therefore, integrating RAG is an effective solution to ensure the accuracy and reliability of the answers. In this project, we implement a RAG system that integrates document retrieval with the LLM to generate answers by jointly leveraging the model's parametric knowledge and non-parametric evidence retrieved from relevant documents. The system is deployed and tested using original data that we collected and processed from various sources within the CSE¹ Faculty. The results show that the system is able to generate responses that adequately satisfy users' information needs, indicating that the RAG approach is suitable for our system development direction.

¹Computer Science and Engineering

Chapter 1

Introduction

*This chapter presents the rationale for choosing the topic and provides an overview of the research project. We begin with the **motivation**, discussing the difficulties faced by learners in accessing academic materials. The **problem statement** outlines the primary objective of developing an intelligent question-answering system for CSE students at HCMUT, focusing on rapid information retrieval, high accuracy, and maintainable architecture. Finally, the **thesis structure** provides an overview of the report's organization across six chapters.*

1.1 Motivation

In the current educational context, learners are increasingly required to take an active role in their learning process, with most learning activities taking place through self-study and independent research. Instead of relying only on classroom lectures, students often need to read documents, lecture slides, and course syllabus to understand essential course content.

However, academic materials are often extensive and presented in various formats, which makes the process of searching for important information more difficult. In many cases, students have to manually read and review entire documents to identify necessary information such as key topics, prerequisites, or assessment and grading methods. This process is not only time-consuming but also inefficient, especially when learners need to quickly access accurate and specific information.

Another important reason is that academic materials are often designed to present the entire course content in a general structure, rather than focusing on specific issues or questions that arise from learners. These materials are typically created to serve general teaching purposes, which causes important information to be scattered across different sections. As a result, students face difficulties in quickly locating the information needed for specific learning tasks, especially in situations where information must be accessed within a limited time.

From these issues, it is clear that accessing and using information from learning materials still has many limitations for students. Manually reading a large amount of documents is not only time-consuming but also reduces learning efficiency, especially when information needs to be found quickly. Therefore, developing a system that can effectively support academic information retrieval while meeting learners' needs is necessary and forms the basis for defining the research objectives of this project.

1.2 Problem statement

The primary objective of this project is to develop an intelligent question-answering system specifically designed to assist students within the Computer Science and Engineering (CSE) faculty at HCMUT in efficiently retrieving accurate and reliable course-related information. The system provides automated question-answering capabilities that enable students to quickly access course information including credit hours, prerequisites, course content summaries, and assessment details. The system offers high-quality answer generation with strong grounding in source materials, effective retrieval of relevant course information, efficient query processing to support rapid information access, and multi-turn conversation support with context memory to enable natural follow-up questions essential for academic applications.

The first objective is to ensure high-quality answer generation with strong semantic relevance to user queries and high faithfulness to source materials. The system must generate answers that are semantically aligned with user queries while being

grounded in retrieved course materials, minimizing hallucination risks and ensuring factual accuracy essential for educational applications.

The second objective focuses on achieving high retrieval performance to ensure that relevant course information is accurately identified and ranked. The system must effectively retrieve semantically relevant chunks that match user query intent, with particular emphasis on identifying the most relevant chunk at the top position to support high-quality answer generation.

The third objective is to evaluate and optimize the impact of query rewriting techniques on overall system performance. The system employs query rewriting to improve semantic matching between user queries and document content, particularly for ambiguous or conversational queries. This objective involves analyzing how query rewriting affects answer quality improvement, retrieval quality improvement, and performance across different query types, enabling identification of optimal query processing strategies for different scenarios.

The fourth objective is to achieve high system performance in terms of computational efficiency and scalability. The system must process queries with low latency and high throughput, enabling rapid information access while maintaining quality standards. Performance is evaluated across different query types (short ambiguous queries, course information queries, specific queries, and full questions) to ensure consistent responsiveness across diverse user query patterns. This objective is measured through average query latency, throughput, and performance breakdown by query type.

The fifth objective is to ensure effective conversation context memory to support natural multi-turn interactions. The system must maintain conversation history (10 most recent messages) and effectively utilize context from previous conversation turns when generating answers for follow-up questions. This capability enables users to ask contextual questions that reference earlier parts of the conversation, creating a more natural and intuitive interaction experience.

1.3 Thesis structure

This Programming Integration Project consists of six chapters in total. Chapter 1 (Introduction), which includes the motivation, problem statement, and this thesis structure overview, has been presented above. The remaining five chapters are organized as follows:

- **Chapter 2 (Theoretical Background)** provides foundational knowledge essential for understanding the RAG system, including Large Language Models (LLMs) architecture and capabilities, Retrieval-Augmented Generation (RAG) architecture, BERT-based embeddings, vector similarity search using FAISS and cross-encoder reranking with BGE Reranker.
- **Chapter 3 (Methods)** describes the methodology for building the RAG system, including functional and non-functional requirements analysis, system architecture design with three main pipelines (Preprocessing, Embedding & Index, Query Pipeline), detailed component specifications (document processing, embedding generation, indexing, retrieval, reranking, and LLM integration), and technology stack selection.
- **Chapter 4 (Implementation and Testing)** describes the implementation of the proposed system based on the designed architecture and methodology. This chapter focuses on how the main components and processing pipelines are realized in practice to construct a complete retrieval-augmented question answering system.
- **Chapter 5 (Experiments)** presents the experimental evaluation of the implemented system. This chapter analyzes the system performance and effectiveness through a series of experiments conducted under different configurations, providing insights into how well the system meets the defined objectives.
- **Chapter 6 (Conclusion)** summarizes the project achievements, evaluates the extent to which each objective was reached, presents key findings from the eval-

uation, and outlines future directions for query processing improvements and system infrastructure enhancements.

Chapter 2

Theoretical Background

*This chapter provides the theoretical foundations required to understand the proposed RAG-based system. It introduces the core concepts underlying the system design, including **Large Language Models (LLMs)**, **Retrieval-Augmented Generation (RAG)**, **semantic embeddings**, **vector search and reranking mechanisms**. These topics are presented in a progressive manner, starting from fundamental retrieval principles and moving toward advanced components used in the system implementation, thereby establishing a clear conceptual basis for the subsequent methodology and implementation chapters.*

2.1 Large Language Models

Large Language Models that are pre-trained on large-scale corpus have demonstrated significant potential in various natural language processing tasks . Most LLMs are based on the Transformer design, which consists of encoder and decoder modules that are empowered by a self-attention mechanism. Based on their architecture structure, LLMs can be classified into three categories: encoder-only LLMs, encoder-decoder LLMs, and decoder-only LLMs. Figure 2.1 summarizes several representative LLMs with different model architectures, model sizes, and open-source availabilities.

Encoder-only LLMs are a type of Large Language Models that use only the en-

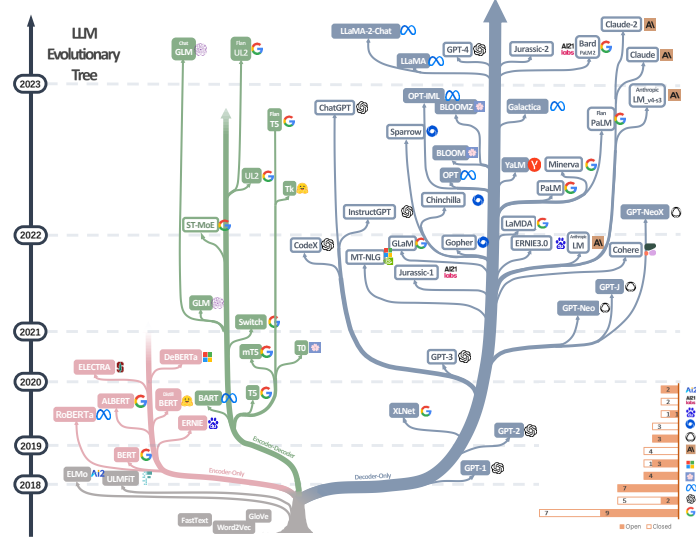


Figure 2.1: The evolutionary tree of modern LLMs traces the development of language models in recent years [1]

coder to encode the sentence and understand the relationships between words. These models are typically pre-trained by predicting the mask words in an input sentence using large-scale corpus. To resolve downstream tasks, such as text classification and named entity recognition, an extra prediction head is added to encoder-only LLMs like BERT [2], RoBERTa [3], ALBERT [4], and ELECTRA [5]. These models are most effective for tasks that require understanding the entire sentence.

Encoder-decoder LLMs adopt both the encoder and decoder module. The encoder module is responsible for encoding the input sentence into a hidden space, and the decoder is used to generate the target output text. The training strategies in encoder-decoder LLMs can be more flexible. Encoder-decoder LLMs are able to directly resolve tasks that generate sentences based on some context, such as summariaztion, translation, and question answering.

Decoder-only LLMs only adopt the decoder module to generate target output text. These models are trained to predict the next word in a sentence. They can perform downstream tasks with few examples or simple instructions without adding predic-

tion heads or fine-tuning [6]. Many state-of-the-art LLMs follow the decoder-only architecture. Recently Llama-2 [7], Qwen [8], Alpaca¹ and Vicuna² are released as open-source decoder-only LLMs. These models are finetuned based on LLaMA [9] and achieve comparable performance with ChatGPT [10] and GPT-4

2.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a framework that combines document retrieval with text generation to support knowledge-intensive natural language processing tasks. Instead of relying solely on parametric knowledge, RAG retrieves relevant documents from an external corpus and incorporates them into the generation process. This approach was first introduced by Lewis et al. [11] and has since been widely adopted in systems that require accurate and up-to-date information access.

2.2.1 RAG pipeline

The RAG framework organizes the generation process into two main stages: **retrieval** and **generation**. Given an input query, the retrieval component first selects a set of relevant textual passages from an external document collection. These retrieved passages are then provided as contextual input to the generation component, which produces the final output sequence.

At a high level, the RAG pipeline consists of several sequential steps. The pipeline begins by **receiving a user query as input**. This query represents the information need expressed by the user and serves as the initial signal for subsequent processing stages. An accurate representation of the input query is essential, as it directly influences both the retrieval effectiveness and the quality of the generated response. Following query input, the system **retrieves a set of relevant documents or passages from an external knowledge source**. This retrieval step enables the system to access non-parametric knowledge stored outside the language model parameters. By select-

¹<https://crfm.stanford.edu/2023/03/13/alpaca.html>

²<https://lmsys.org/blog/2023-03-30-vicuna/>

ing relevant passages based on semantic similarity, the retriever provides contextual evidence that supports downstream answer generation.

In the final stage of the pipeline, **the text generation process is conditioned on both the original query and the retrieved documents**. By incorporating retrieved contextual information into the generation process, the system produces responses that are grounded in external evidence rather than relying solely on internal model knowledge. This retrieve-then-generate pipeline allows the system to dynamically access information at inference time, improving factual consistency and adaptability to domain-specific knowledge.

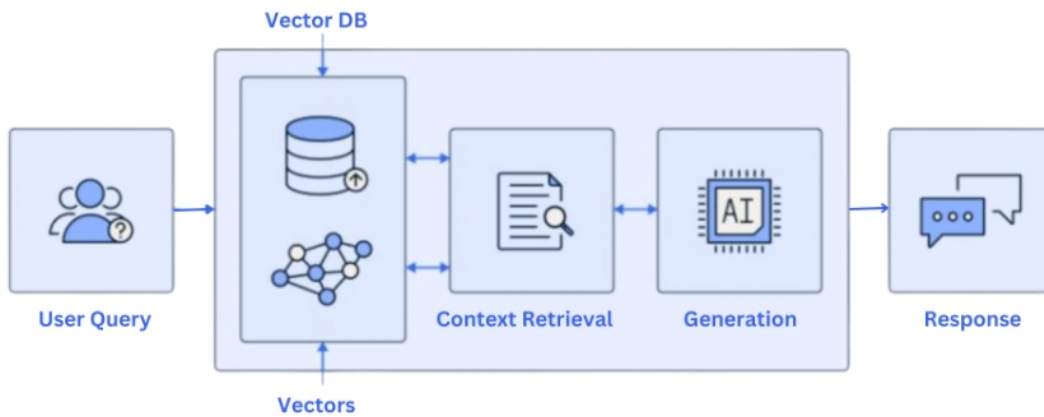


Figure 2.2: Conceptual RAG pipeline adapted from Lewis et al. (2020).

As illustrated in Figure 2.2, this pipeline allows the system to dynamically access information at inference time, ensuring that generated responses are grounded in retrieved evidence rather than relying entirely on internal model knowledge.

2.2.2 RAG Components

Within the RAG framework, retrieval and generation serve distinct but complementary roles. Each component contributes to the overall effectiveness of the system by addressing different aspects of the question-answering process.

The **retrieval component** is responsible for identifying and selecting documents or passages that are semantically relevant to the input query. By encoding both the query and candidate documents into a shared representation space, the retriever estimates se-

mantic similarity and ranks documents accordingly. This component plays a critical role in ensuring that the generation component has access to sufficient and informative contextual evidence, which is essential for producing accurate and relevant answers, particularly in knowledge-intensive tasks.

The **generation component** produces the final output by conditioning on both the original user query and the retrieved documents. Rather than generating responses solely based on internal model representations, the generator leverages external contextual evidence to construct answers that are more factual and context-aware. By grounding the generation process in retrieved source materials, this component helps reduce the risk of unsupported or hallucinated content while maintaining coherence and relevance in the generated responses.

The clear separation between retrieval and generation enables a modular system design, allowing each component to be improved or replaced independently. This modularity supports flexible system development and facilitates experimentation with different retrieval or generation strategies without affecting the overall framework.

2.2.3 Key Characteristics of RAG Frameworks

RAG-based systems exhibit several important characteristics that distinguish them from purely parametric language models. These characteristics highlight how retrieval-augmented architectures address the limitations of relying solely on internal model knowledge for knowledge-intensive tasks.

The first key of RAG is the **integration of parametric and non-parametric memory**. Parametric memory refers to knowledge implicitly stored within the parameters of a language model, while non-parametric memory consists of an external document collection that can be explicitly accessed at inference time. By combining these two forms of memory, RAG enables models to leverage both learned language representations and external knowledge sources, improving their ability to handle queries that require factual or domain-specific information.

Another important characteristic is **dynamic knowledge access**. In RAG frameworks,

the underlying document collection can be updated, expanded, or replaced without retraining the generation model. This property allows the system to remain adaptable to changes in knowledge over time and supports efficient maintenance, particularly in academic environments where learning materials are frequently updated.

RAG also provides **improved factual grounding** compared to purely parametric language models. Since generated outputs are conditioned on retrieved evidence, responses are more closely aligned with source materials rather than relying solely on implicit model memory. This grounding mechanism helps reduce factual errors and improves the reliability of generated answers, which is especially important for educational and knowledge-intensive applications.

Finally, RAG frameworks offer **flexibility across tasks**. The same retrieval-augmented architecture can be applied to a wide range of knowledge-intensive generation and reasoning tasks without requiring substantial architectural modifications. This flexibility makes RAG suitable for various applications that demand accurate information retrieval and contextualized text generation.

These characteristics make RAG well aligned with the objectives defined in Section 1.2, as it supports grounded answer generation, effective retrieval of course-related information, and efficient handling of diverse and conversational user queries.

2.3 BERT-based Embeddings for Semantic Retrieval

BERT (Bidirectional Encoder Representations from Transformers) [2] has become a foundational model for generating dense embeddings in information retrieval systems. Unlike traditional word-level embeddings, BERT produces contextualized representations that capture bidirectional context, making it particularly suitable for semantic similarity tasks.

2.3.1 Why BERT for Embeddings

BERT is chosen for embedding generation due to several key advantages:

- **Bidirectional context:** Unlike unidirectional language models, BERT processes text in both directions simultaneously, enabling richer semantic understanding by considering both preceding and following context for each token.
- **Pre-trained knowledge:** BERT is pre-trained on large corpora using masked language modeling and next sentence prediction objectives, encoding substantial linguistic and world knowledge that transfers well to downstream tasks.
- **Task adaptability:** Through fine-tuning or feature extraction, BERT embeddings can be adapted for various retrieval tasks without requiring task-specific architectural changes.

However, vanilla BERT is not optimized for producing sentence-level embeddings suitable for similarity search. Sentence-BERT (SBERT) [12] addresses this limitation by fine-tuning BERT with siamese and triplet network structures on sentence-pair datasets, enabling efficient semantic similarity computation.

2.3.2 Pooling and Normalization Strategies

Converting BERT’s token-level outputs to fixed-size sentence embeddings requires pooling strategies:

- **Mean pooling:** Computes the average of all token embeddings (excluding special tokens). This approach captures information from all tokens and is empirically shown to outperform CLS token pooling for semantic similarity tasks [12].
- **CLS token pooling:** Uses the [CLS] token’s embedding as the sentence representation. While designed for classification tasks, it often underperforms mean pooling for retrieval due to limited semantic information.

- **Max pooling:** Takes the maximum value across tokens for each dimension. Less commonly used for semantic tasks as it may emphasize outlier features.

Normalization is crucial for embedding quality. L2 normalization (unit vector scaling) is commonly applied to embeddings before similarity computation. This normalization:

- Reduces the impact of embedding magnitude on similarity scores
- Improves training stability and convergence
- Makes cosine similarity equivalent to dot product for normalized vectors

2.3.3 Similarity Metrics

The choice of similarity metric directly impacts retrieval performance:

- **Cosine similarity:** Measures the cosine of the angle between two vectors: $\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|}$. For L2-normalized embeddings, this reduces to dot product. Cosine similarity is scale-invariant and widely used for semantic retrieval.
- **Inner product (dot product):** Computes $\mathbf{u} \cdot \mathbf{v} = \sum_i u_i v_i$. When embeddings are normalized, inner product equals cosine similarity. Inner product is computationally efficient and directly optimizable in vector search libraries like FAISS.
- **Euclidean distance:** Measures L_2 distance: $\|\mathbf{u} - \mathbf{v}\|_2$. Less common for semantic similarity as it is sensitive to vector magnitude.

For normalized embeddings, cosine similarity and inner product are equivalent and both are commonly used in production systems.

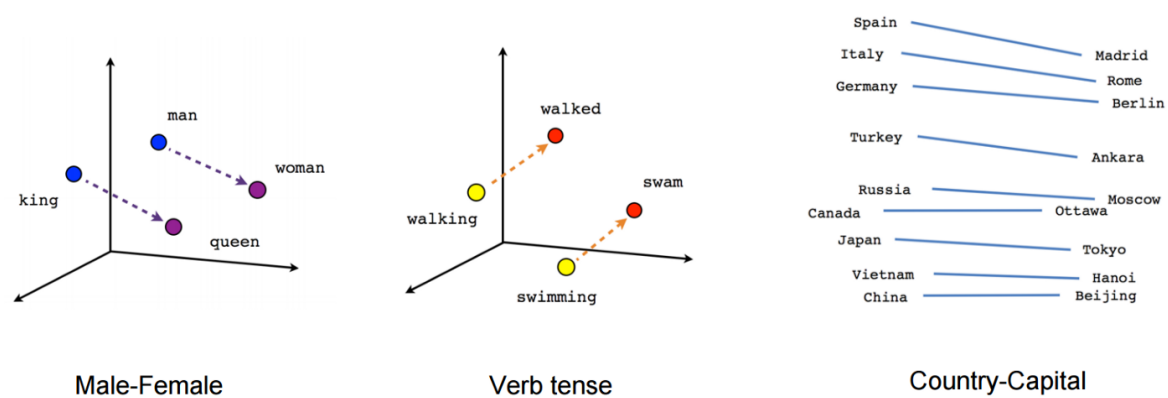


Figure 2.3: Examples of semantic relationships captured in embedding space, illustrating gender relations, verb tense transformations, and country–capital analogies.

2.3.4 Embedding Dimension and Impact on Recall

The embedding dimension (typically 768 for BERT-base, 1024 for BERT-large) affects both retrieval quality and computational efficiency:

- **Higher dimensions** can capture more nuanced semantic information, potentially improving recall for complex queries. However, they increase storage requirements and search latency.
- **Lower dimensions** (achieved via dimensionality reduction) reduce computational cost but may lose semantic expressiveness, potentially decreasing recall.
- **Recall trade-offs:** In approximate nearest neighbor search (e.g., FAISS), higher-dimensional embeddings require more memory and may need larger index parameters (e.g., more centroids in IVF) to maintain recall. The choice of dimension balances between semantic expressiveness and search efficiency.

Empirical studies show that BERT-base embeddings (768 dimensions) with mean pooling and L2 normalization provide a good balance between quality and efficiency for most retrieval tasks, achieving strong recall while remaining computationally tractable for large-scale vector databases.

2.3.5 Lightweight Alternatives: MiniLM

While BERT-base and BERT-large provide high-quality embeddings, their computational requirements (110M and 340M parameters respectively) can be prohibitive for resource-constrained deployments. MiniLM [13] addresses this limitation through knowledge distillation, compressing larger transformer models into smaller, faster variants while preserving semantic quality.

MiniLM architecture: MiniLM uses deep self-attention distillation to transfer knowledge from teacher models (e.g., BERT-base) to student models with fewer layers and hidden dimensions. The distillation process focuses on matching attention distributions and hidden states, enabling the student to capture essential semantic patterns.

all-MiniLM-L6-v2: The all-MiniLM-L6-v2 model, part of the Sentence-Transformers library, is a distilled version optimized for sentence embeddings. Key characteristics:

- **Compact size:** 22.7M parameters (compared to 110M for BERT-base), enabling faster inference and lower memory footprint.
- **Lower dimensionality:** 384-dimensional embeddings (vs. 768 for BERT-base), reducing storage requirements and search latency in vector databases.
- **Performance:** Despite being smaller, MiniLM maintains competitive semantic quality on benchmark tasks, often achieving 95–98% of BERT-base performance with 2–3x faster inference.

Trade-offs and selection criteria: The choice between BERT-base and MiniLM involves balancing:

- **Quality vs. efficiency:** BERT-base offers marginally better semantic understanding for complex queries, while MiniLM provides faster batch processing and lower resource consumption.
- **Recall impact:** For most retrieval tasks, the recall difference between 384-dimensional MiniLM and 768-dimensional BERT-base embeddings is minimal

(typically $<2\%$), making MiniLM suitable for production systems prioritizing latency and cost.

- **Scalability:** Lower-dimensional embeddings enable more efficient FAISS indexing, allowing larger candidate sets to be searched within the same memory budget.

In practice, MiniLM is often preferred for production RAG systems where inference speed and resource efficiency are critical, while maintaining acceptable retrieval quality. The 384-dimensional embeddings provide sufficient semantic expressiveness for most domain-specific retrieval tasks while enabling real-time query processing.

2.4 Vector Search

Vector search [14] constitutes a fundamental paradigm for semantic information retrieval, in which documents and queries are represented as points in a continuous vector space and relevance is determined by geometric proximity rather than lexical overlap. Unlike traditional keyword-based retrieval methods, vector search enables the retrieval of semantically related content even when there is little or no surface-level term matching, making it particularly suitable for modern language model-based systems. From a conceptual perspective, vector search should not be viewed merely as a nearest neighbor algorithm, but rather as a retrieval abstraction that integrates representation learning, similarity metrics, indexing strategies, and system-level trade-offs. This abstraction forms the backbone of dense retrieval systems and plays a critical role in Retrieval-Augmented Generation (RAG) pipelines [11].

2.4.1 Vector-Based Retrieval Mechanism

In vector search, each document or text segment is mapped to a dense vector

$$\mathbf{v}_i \in \mathbb{R}^d$$

using an embedding model [12], while a user query is encoded into a query vector

$$\mathbf{q} \in \mathbb{R}^d$$

within the same vector space.

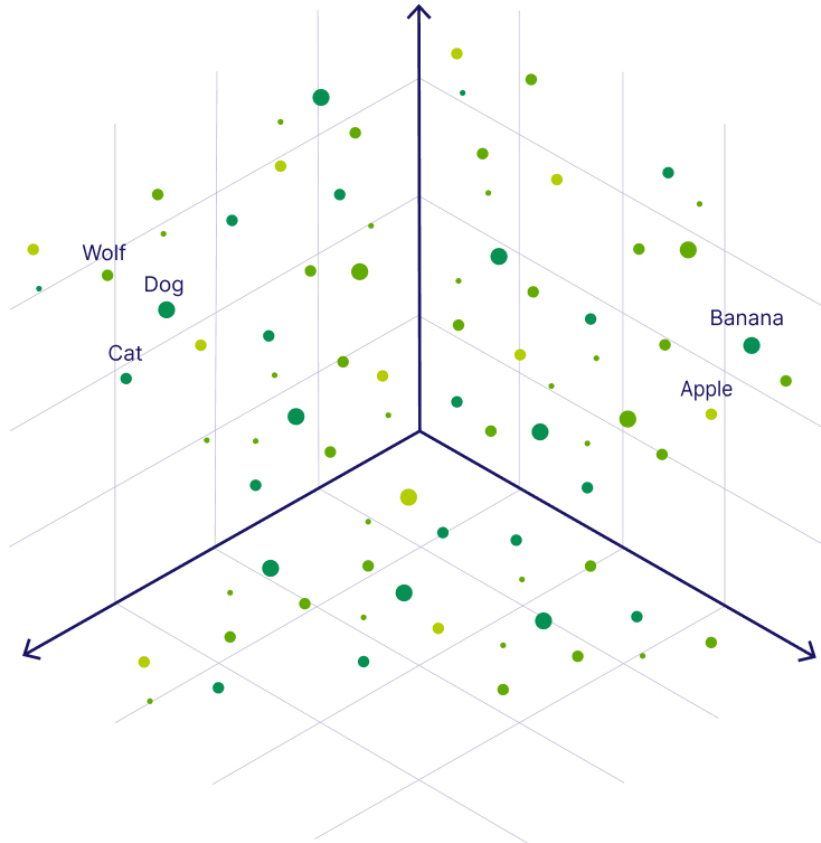


Figure 2.4: Illustration of documents and queries represented in a shared embedding space, where semantic relevance is determined by geometric proximity.

The objective of the retrieval process is to identify the set of document vectors that

are most similar to the query vector. Formally, this can be expressed as:

$$\text{top-k}(\mathbf{q}) = \arg \max_{\mathbf{v}_i} s(\mathbf{q}, \mathbf{v}_i),$$

where $s()$ denotes a similarity function.

Common similarity measures include cosine similarity, inner product, and Euclidean distance. In practice, cosine similarity is widely adopted for semantic retrieval tasks due to its robustness to vector magnitude. When vectors are L2-normalized, cosine similarity becomes equivalent to the inner product:

$$\cos(\mathbf{q}, \mathbf{v}) = \frac{\mathbf{q} \cdot \mathbf{v}}{\|\mathbf{q}\| \|\mathbf{v}\|}.$$

Because embeddings generated by modern language models often reside in high-dimensional spaces, exhaustive linear search can become computationally expensive. As a result, vector search systems employ various indexing strategies and search algorithms, including exact search and approximate nearest neighbor methods, to balance retrieval accuracy and efficiency.

2.4.2 Vector database

With the growing adoption of vector-based retrieval, a variety of vector databases have been developed to support large-scale storage and search over embedding vectors. These systems typically integrate vector search capabilities with persistent storage, metadata management, conditional filtering, and distributed deployment, targeting production environments with dynamic and large-scale data.

Representative vector database systems include *Qdrant* [15], which emphasizes approximate nearest neighbor search combined with payload-based filtering, *Milvus* [16], which provides a distributed architecture for large-scale vector data management, and *Weaviate* [17], which integrates vector search with schema-aware data modeling and API-based access. These systems illustrate a service-oriented approach in which vector search is tightly coupled with database-level functionalities.

In parallel, vector search libraries focus primarily on efficient similarity computation and indexing, without providing full database functionalities. Rather than serving as standalone data management systems, these libraries act as core computational components that can be embedded into broader retrieval pipelines.

FAISS is a representative example of vector search libraries, designed specifically for efficient similarity search over dense vectors in high-dimensional spaces. In the context of RAG systems, FAISS is often referenced as a canonical implementation of library-based vector search, while other vector database systems exemplify service-oriented approaches tailored for large-scale production use .

2.4.3 Role of Vector Search in Retrieval-Augmented Generation (RAG)

Within a Retrieval-Augmented Generation framework [11], vector search typically functions as the first-stage retriever. Its primary role is to identify a set of candidate text segments that exhibit high semantic relevance to a given query.

The effectiveness of this retrieval stage has a direct impact on subsequent components, such as reranking mechanisms and language model-based answer generation. A vector search system with high recall helps ensure that relevant contextual information is made available to downstream models, thereby supporting more accurate and contextually grounded responses.

From a theoretical perspective, vector search enables semantic matching in a continuous embedding space, allowing retrieval decisions to be based on meaning rather than exact lexical overlap. This property is particularly important in RAG systems, where queries and relevant documents may share semantic intent despite differing surface forms.

For this reason, vector search is widely regarded as a foundational component in RAG architectures, serving as the critical link between semantic representation learning and generative language modeling.

2.5 Reranking in Information Retrieval

Reranking is a two-stage retrieval strategy that refines initial retrieval results by applying a more sophisticated scoring model to a small candidate set [18]. In the first stage, a fast retriever (typically a bi-encoder) retrieves a large set of candidates (e.g., top- k with $k = 100$). In the second stage, a reranker (typically a cross-encoder) re-scores these candidates to produce the final ranked list.

2.5.1 Cross-Encoder vs Bi-Encoder Architectures

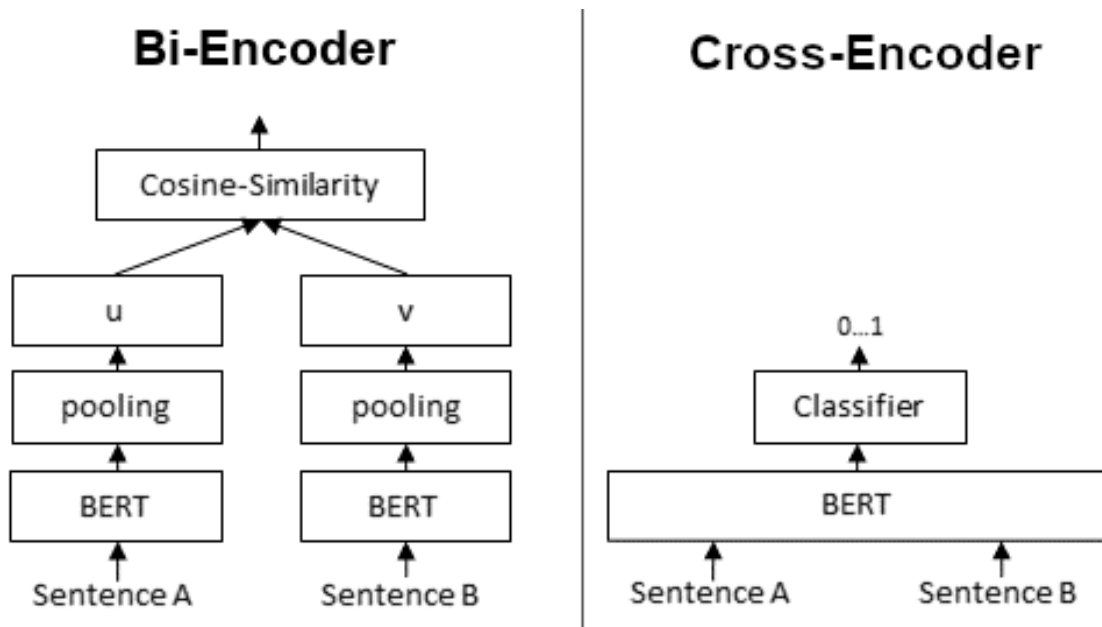


Figure 2.5: Bi-Encoder and Cross-Encoder architectures comparison [19]

The choice between cross-encoder and bi-encoder architectures represents a fundamental trade-off between accuracy and efficiency:

- **Bi-encoder** (used in initial retrieval): Query and document are encoded separately into dense vectors. Similarity is computed via dot product or cosine similarity. This enables efficient approximate nearest neighbor search (e.g., using FAISS, Qdrant) with sub-linear time complexity, making it suitable for large-scale retrieval. However, bi-encoders cannot model fine-grained query–document interactions.

- **Cross-encoder** (used in reranking): Query and document are concatenated and fed into a single transformer model, enabling joint attention over both inputs. This allows the model to capture complex semantic interactions and fine-grained relevance signals, typically achieving higher accuracy than bi-encoders. However, cross-encoders require computing scores for each query–document pair independently, making them computationally expensive for large candidate sets.

The two-stage approach leverages the strengths of both: bi-encoders provide fast candidate retrieval, while cross-encoders provide accurate final ranking on a small subset.

2.5.2 When Reranking is Needed

Reranking becomes particularly valuable when:

- The initial retriever has high recall but lower precision, requiring refinement to identify the most relevant passages.
- The downstream task (e.g., LLM generation) benefits from higher-quality context, where even small improvements in top- k relevance can significantly impact answer quality.
- Computational budget allows for the additional latency cost of reranking a small candidate set (typically 10–50 items).

2.5.3 BGE Reranker

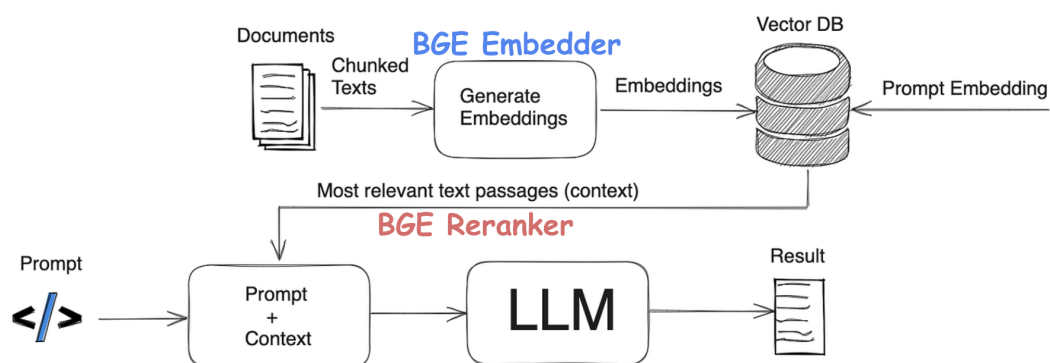


Figure 2.6: BGE embedder and reranker in an RAG pipeline [19]

The BGE (BAAI General Embedding) reranker is a cross-encoder model developed by the Beijing Academy of Artificial Intelligence (BAAI) [19]. Built on XLM-RoBERTa architecture, it is specifically designed for reranking tasks in both English and Chinese. The model employs a cross-encoder architecture where query and passage are concatenated and processed jointly, enabling fine-grained relevance scoring. BGE reranker models (base and large variants) have demonstrated state-of-the-art performance on various retrieval benchmarks, making them suitable for production RAG systems where accuracy is prioritized over minimal latency.

Chapter 3

Methods

In Chapter 3, it presents the methodology for building the RAG system [11], including requirements analysis, system design, and detailed implementation of preprocessing, embedding, indexing, and LLM-powered query optimization.

3.1 Requirements Analysis

This section analyzes the functional and non-functional requirements for developing a RAG-based questioning-answering system [11] tailored for Computer Science course materials at HCMUT.

3.1.1 Functional Requirements

The system provides semantic search functionality across course materials from multiple sources, including the HCMUT LMS [20] and Coursewave [21]. It retrieves relevant content based on semantic similarity and provides source citations that link retrieved content to original document sources. The system supports both per-course and cross-course retrieval capabilities, allowing users to search within a specific course or across all available courses.

The system accepts natural language questions in English from users. It retrieves relevant document chunks based on semantic similarity and reranks them to improve relevance, providing source citations for each response.

The system generates answers based on the context obtained from retrieved chunks, supporting multiple LLM providers with configurable models. Generated answers include proper citation of sources and handle cases where no relevant context is found by providing appropriate feedback.

The system provides course selection functionality, allowing users to specify a particular course to scope their queries or search across all available courses. It maintains conversation history and context across multiple turns, enabling users to ask follow-up questions that reference previous messages in the conversation. The conversation context is utilized during answer generation to provide coherent and contextually relevant responses.

The user interface features a web-based chat interface for user interactions, displaying conversation history and enabling follow-up questions. It shows source citations for each answer and provides an API endpoint to allow programmatic access.

3.1.2 Non-Functional Requirements

The system must generate answers with strong semantic relevance to user queries and high faithfulness to source materials. Answers must be semantically aligned with user queries while being grounded in retrieved course materials, minimizing hallucination risks and ensuring factual accuracy essential for educational applications.

The system must effectively retrieve semantically relevant chunks that match user query intent, with particular emphasis on identifying the most relevant chunk at the top position. It must accurately identify and rank relevant course information to support high-quality answer generation.

The system architecture must support configurable query processing strategies, enabling analysis of how query rewriting affects answer quality improvement, retrieval quality improvement, and performance across different query types. This allows identification of optimal approaches for different scenarios.

The system must process queries with low latency and high throughput, enabling rapid information access while maintaining quality standards. It is expected to effi-

ciently retrieve relevant chunks within 500 milliseconds for queries against a corpus of over 10,000 chunks, generating answers below 30 seconds. Performance must be consistent across different query types to ensure responsive user experience. The embedding model will produce embeddings of 384 dimensions, balancing quality and efficiency.

The system must maintain conversation history (10 most recent messages) and effectively utilize context from previous conversation turns when generating answers for follow-up questions. It must support natural multi-turn interactions, enabling users to ask contextual questions that reference earlier parts of the conversation.

3.1.3 Data Requirements

The input data will comprise course syllabi and lecture materials in PDF and PPTX formats, allowing for the extraction of chapter numbers, page indices, and hierarchical content organization. The output will include structured JSON files for parsed content, vector embeddings stored in FAISS indices [14] with the associated metadata, and generated answers formatted with source citations for API responses.

To maintain data quality, the OCR process [22] will focus on achieving sufficient accuracy for extracting readable text from scanned documents, while the chunking strategy will aim for semantic coherence within the chunks. Metadata must also be complete and accurate to ensure proper source attribution.

3.1.4 Constraints

The system will be designed to run efficiently on CPU without requiring GPU resources for inference. It will be focused specifically on Computer Science course materials at HCMUT, with no need for generalization to other domains. The knowledge base will be kept static, and real-time updates from external sources will not be supported. The availability of LLM APIs will rely on external services (like Gemini API) or local setups (such as Ollama or Qwen [8]).

3.2 System Design

This section presents the overall architecture and design of the RAG system for CSE course materials. The system is designed as a modular pipeline with distinct stages for data processing, indexing, and query answering.

3.2.1 System Architecture Overview

Figure 3.1 illustrates the complete system architecture with all two main pipelines and their interactions.

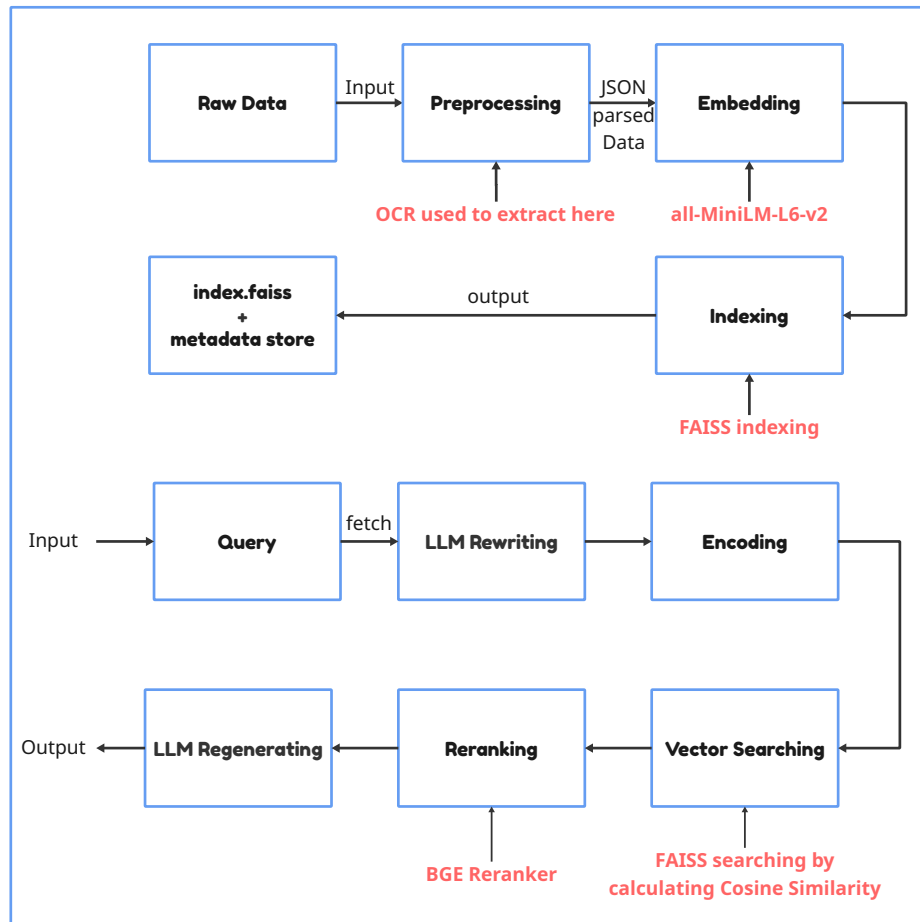


Figure 3.1: Complete system architecture showing two **main pipelines**: Preprocessing and Embedding & Index, and Query Pipeline → API/FE. The diagram illustrates an abbreviated data flow from document ingestion through indexing to query processing and response generation.

The system follows a two-stage architecture.

Preprocessing and Embedding & Index Pipeline. The pipeline first processes raw documents, extracts text using OCR [22], parses structured information, and chunks text for indexing. The pipeline then converts text chunks to vector embeddings, normalizes them, and stores them in FAISS indices [14] with metadata. This offline pipeline addresses the functional requirement for semantic search across multiple course materials by creating searchable vector representations of documents from HCMUT LMS and Coursewave sources. The use of 384-dimensional embeddings directly satisfies the non-functional requirement for balancing quality and efficiency, while the FAISS IndexFlatIP implementation enables retrieval within 500 milliseconds for corpora exceeding 10,000 chunks, meeting the performance requirement. The metadata storage mechanism ensures proper source attribution, fulfilling the functional requirement for source citations linking to original documents.

Query Pipeline. It handles user queries through API/Frontend, retrieves relevant chunks, reranks results [19], and generates answers by LLM [1]. This runtime pipeline addresses multiple functional requirements: it accepts natural language questions in English, retrieves relevant chunks based on semantic similarity, and generates contextually grounded answers. The two-stage retrieval approach (FAISS search followed by BGE reranking) directly satisfies the non-functional requirements for retrieval effectiveness by identifying the most relevant chunks at top positions and improving overall Query-Chunk Semantic Similarity scores. The integration of conversation history storage and context formatting components fulfills the functional requirement for maintaining conversation context across multiple turns (10 most recent messages) and the non-functional requirement for effective multi-turn interaction support. The LLM Client abstraction supporting multiple providers (Gemini, Ollama, Qwen) satisfies the functional requirement for configurable LLM providers, while the optional Query Rewriter component addresses the non-functional requirement for configurable query processing strategies, enabling analysis of query rewriting effects on answer quality and retrieval performance. The response generation with source citations ensures

answers maintain high faithfulness to source materials, meeting the non-functional requirement for minimizing hallucination risks in educational applications.

3.2.2 Component Design

3.2.2.1 Preprocessing Module

The preprocessing module handles document ingestion and text extraction, addressing the fundamental challenge of transforming heterogeneous course materials into a unified, searchable format. The design tackles several critical problems: diverse document formats (PDF, DOCX, PPTX) require specialized conversion and extraction techniques to ensure no information is lost during processing; scanned documents and image-based slides necessitate robust OCR capabilities to extract readable text with sufficient accuracy; maintaining semantic coherence across document boundaries is essential for meaningful retrieval, requiring intelligent chunking strategies that preserve context; and educational applications demand complete traceability from retrieved content back to original sources, necessitating comprehensive metadata attachment throughout the pipeline. The module's multi-stage architecture systematically addresses these challenges through specialized components that work in concert to produce high-quality, semantically coherent text chunks enriched with structural metadata. Figure 3.2 shows the detailed preprocessing pipeline.

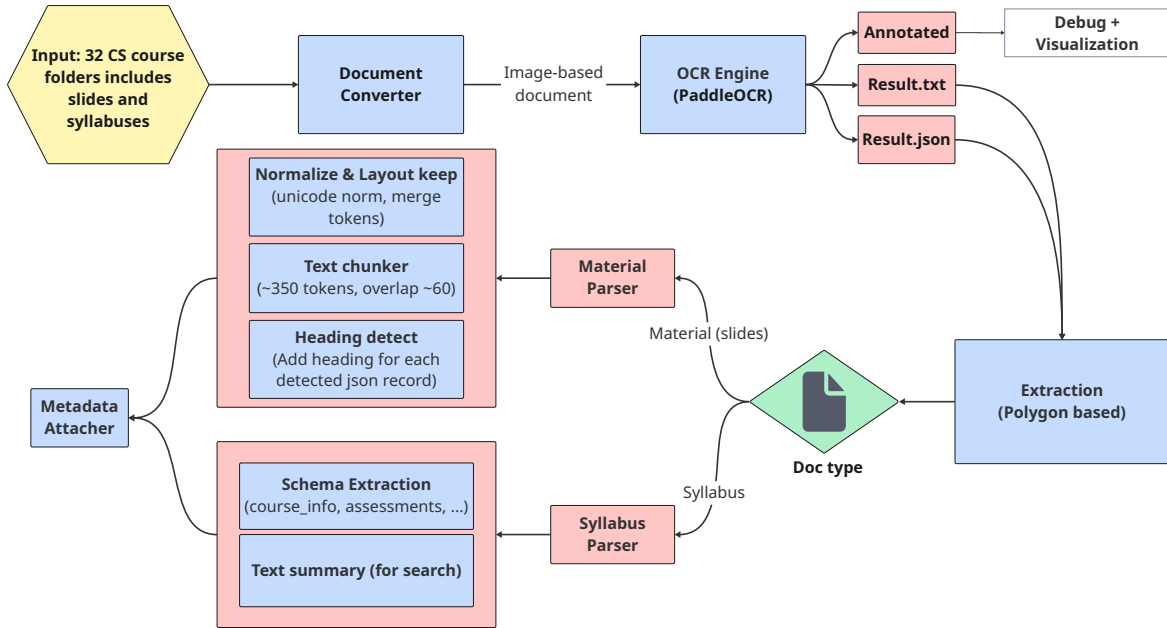


Figure 3.2: Preprocessing pipeline showing document conversion, OCR processing, parsing, and chunking stages.

The preprocessing module consists of several key components, each addressing a specific challenge in the document processing pipeline.

Document Converter. To enable text extraction from diverse document formats, this component converts PDF/DOCX/PPTX files to page images at 220 DPI resolution, creating a uniform input format for OCR processing regardless of the original document structure.

OCR Engine. Addressing the challenge of extracting text from scanned documents and image-based slides, this component utilizes PaddleOCR 3.2 [22], [23] to extract text with bounding box information and confidence scores, enabling accurate text extraction with spatial awareness.

Extraction Module. To isolate relevant text regions and filter out noise from OCR results, this component performs polygon-based text region extraction utilizing bounding box coordinates to identify and isolate text regions, producing annotated outputs (result.txt, result.json) and debug visualizations for quality assurance.

Syllabus Parser. Recognizing that syllabi contain critical structured information distinct from general course content, this component extracts course metadata, grading policies, and learning objectives in a structured format that enables better context understanding.

Material Parser. To maintain the hierarchical organization of lecture materials, this component processes lecture slides to extract chapter information, page indices, and content structure, preserving the pedagogical flow of the original materials.

Text Chunker. Addressing the need to balance retrieval granularity with semantic completeness, this component segments documents into overlapping chunks using a sliding window approach with configurable window size and overlap, ensuring that semantic context is preserved across chunk boundaries and preventing information loss at boundaries.

Metadata Attacher. To enable complete traceability from retrieved content back to source documents, this component enriches extracted text with document metadata including course information, page indices, and structural annotations, ensuring that each text chunk maintains precise attribution to its source document and position.

3.2.2.2 Embedding & Indexing Module

The embedding and indexing modules work together to create searchable vector representations. Figure 3.3 illustrates the embedding generation, normalization, and indexing process.

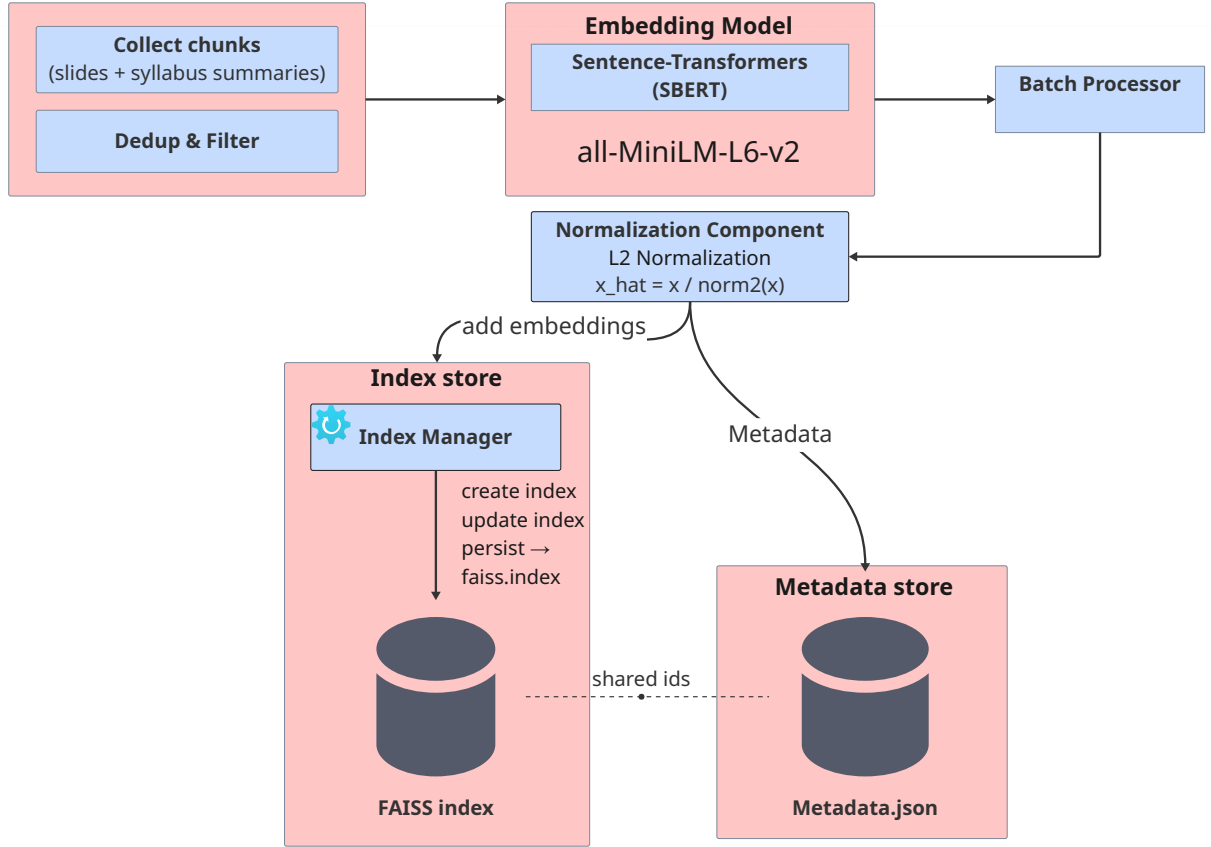


Figure 3.3: Embedding and indexing pipeline showing text-to-embedding conversion, L2 normalization, and storage in FAISS index with metadata.

The embedding module converts text into vector representations through a multi-stage process, transforming semantic meaning into searchable mathematical objects.

Embedding Model. To enable semantic search beyond keyword matching, this component employs sentence-transformers/all-MiniLM-L6-v2 [12], [13] to generate 384-dimensional embeddings that capture semantic meaning of text chunks, producing dense vector representations that position semantically similar content closer together in vector space.

Batch Processor. Addressing the computational challenge of processing thousands of text chunks efficiently, this component processes chunks in batches for efficient embedding generation, optimizing computational resources and throughput by handling multiple chunks simultaneously rather than sequentially.

Normalization Component. To enable efficient similarity computation using inner product operations instead of computationally expensive cosine calculations, this component L2-normalizes the generated embeddings using the formula

$$\hat{x} = x / \|x\|_2$$

to create unit vectors, which mathematically transforms cosine similarity into simple inner product operations. The normalized embeddings and their associated metadata are then stored in parallel through two synchronized pathways.

Index Manager. To ensure persistent storage and efficient lifecycle management of vector indices across multiple courses, this component orchestrates the index lifecycle by creating new FAISS indices, updating them with normalized embeddings, and persisting them to disk as faiss.index files, ensuring data durability and consistent index management.

FAISS Index. To provide fast and precise retrieval from large document collections, this component utilizes IndexFlatIP (Inner Product) [14] for exact similarity search on normalized vectors, employing brute-force inner product computation to provide precise retrieval results with dimension $d = 384$ matching the embedding model output.

Metadata Store. To enable reconstruction of original text and source attribution from retrieved vector IDs, this component maintains JSON mappings in metadata.json from vector IDs to both the actual chunk text content and document metadata including course ID, page index, chapter, and source file information, enabling traceability of retrieved content. The vector IDs are shared between the FAISS Index and Metadata Store, ensuring synchronized access to embeddings and their corresponding text content.

3.2.2.3 Query Pipeline Module

The query pipeline module handles the complete flow from user query to response generation. Figure 3.4 shows the detailed query processing architecture including API backend, reranking, and frontend interaction.

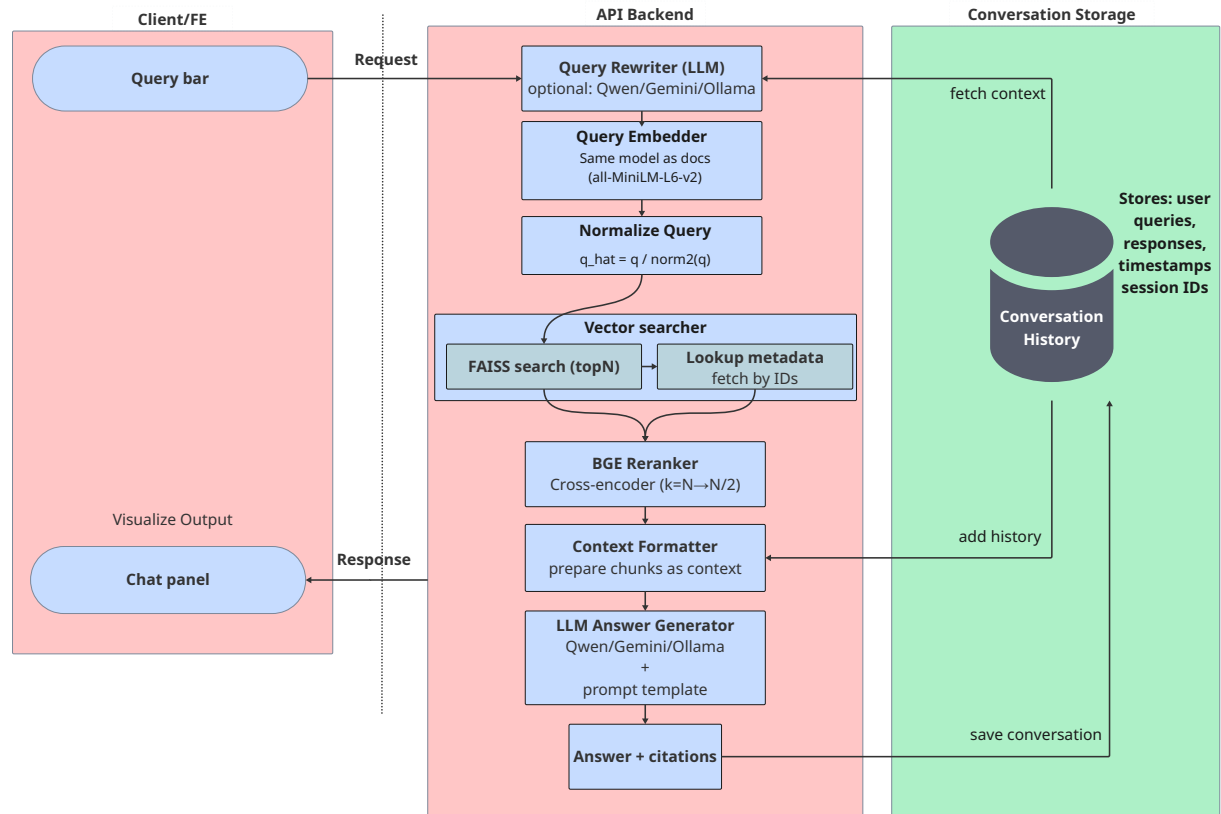


Figure 3.4: Query pipeline showing client/Frontend interaction with API Backend, vector search in Index Store, reranking with BGE Reranker, and response generation.

The query pipeline processes user queries and generates responses through multiple interconnected components, each addressing specific challenges in the retrieval and generation workflow.

Query Rewriter. To improve retrieval quality for ambiguous or poorly-formulated user questions, this component optionally uses an LLM [1], such as Qwen or Gemini [8], to reformulate user queries, transforming vague questions into semantically richer versions that enhance matching with document content. The rewritten query is then passed to the embedding stage.

Query Embedder. To enable semantic comparison between user questions and document content in the same vector space, this component converts the query text to a 384-dimensional embedding vector using the same all-MiniLM-L6-v2 model employed for document embeddings, ensuring consistency in the vector space representation. The generated query embedding is then L2-normalized using the formula $\hat{q} = q/\|q\|_2$ to create a unit vector, enabling cosine similarity computation via inner product operations.

Vector Searcher. To rapidly identify potentially relevant content from large document collections, this component performs top- k similarity search (default $k = 10$) using FAISS [14] with the normalized query embedding converted from Query Embedder, computing cosine similarities via inner product to identify the most relevant document chunks. The searcher retrieves both the similarity scores and the corresponding vector IDs, which are then used to fetch the actual chunk text and metadata from the Metadata Store.

Reranker. To refine initial retrieval results and improve precision by capturing fine-grained query-passage interactions that bi-encoders miss, this component applies the FlagEmbedding cross-encoder (BGE Reranker) [19] to rerank the retrieved chunks by computing fine-grained relevance scores through joint query-passage encoding. The reranker groups passages by (query, passage) pairs, computes cross-encoder scores using $\text{score} = f(\text{query}, \text{text})$, applies softmax normalization for confidence estimation, and returns the top-ranked chunks (typically $k/2$) with the highest confidence scores, significantly improving precision over the initial retrieval.

Context Formatter. To prepare retrieved information in an optimal format for language model consumption and enable multi-turn dialogue understanding, this component prepares the reranked chunks as structured context for the LLM prompt, organizing the retrieved information by concatenating chunk texts with their metadata (course

ID, page index, chapter) in a format optimized for answer generation. Additionally, it retrieves relevant conversation history from the Conversation History Storage and incorporates previous queries and responses into the context, enabling coherent multi-turn interactions. The formatted context is then passed to the answer generation stage.

Conversation History Storage. To enable natural follow-up questions and maintain dialogue coherence across multiple interaction turns, this component maintains a persistent record of user interactions by storing conversation data including user queries, system responses, timestamps, and session IDs in a structured in-memory dictionary format, organized by session IDs as keys. The storage implements automatic cleanup mechanisms to prevent memory leaks by limiting the maximum number of concurrent conversations (default 1,000 sessions) and the maximum messages per conversation (default 10 most recent messages), using a FIFO strategy to remove the oldest conversations when limits are reached. The storage supports three key operations: (1) fetching previous conversation context for the Query Rewriter to understand follow-up questions and resolve pronouns or references, (2) providing conversation history to the Context Formatter to maintain dialogue coherence across multiple turns, and (3) saving new query-response pairs after answer generation to build up the conversation context for future interactions.

LLM Answer Generator. To synthesize natural language answers grounded in retrieved course materials while supporting flexible deployment scenarios, this component leverages a large language model through the LLM Client abstraction, which supports multiple providers including Gemini, Ollama, and Qwen [8], to generate accurate answers. The generator constructs a prompt template that includes the user's original query and the formatted context, then calls the LLM API to produce a natural language response grounded in the retrieved course materials.

Response Generator. To provide complete, traceable answers with proper source

attribution and maintain conversation continuity, this component packages the LLM-generated answer along with source citations (including course names, page numbers, and chunk references) into a structured response format, then saves the complete query-response pair to the Conversation History Storage before returning the response to the user via the API backend and displaying it in the frontend chat panel, completing the query-answer cycle.

3.2.3 API and Frontend Architecture

To serve a production-ready RAG system that balances accessibility, maintainability, and scalability, we designed a three-tier architecture with containerized deployment. This architectural choice addresses several critical deployment challenges: separating user interface concerns from backend logic enables independent development and scaling of frontend and backend components; exposing the RAG pipeline through RESTful APIs allows programmatic access beyond the web interface, supporting future integrations with other applications or services; containerization via Docker Compose ensures consistent deployment across development, testing, and production environments while simplifying dependency management; and the modular structure facilitates easy replacement or upgrade of individual components without disrupting the entire system. The system exposes its RAG capabilities through this three-tier architecture consisting of a React-based frontend, a FastAPI backend, and the core RAG pipeline, separating user interaction concerns from retrieval and generation logic while optimizing for responsive user experience.

React Frontend. It provides an interactive chat interface running on port 5173 where users submit natural language queries and view responses with source citations, communicating with the backend via asynchronous HTTP requests to prevent UI blocking during query processing. The **Chat Interface** component manages message history and user input as the primary interaction point, while the **Message List** displays conversation history with user queries and assistant responses in scrollable format.

FastAPI Backend. It exposes RESTful endpoints on port 8000 for query processing, health checks, and streaming responses, validating requests using Pydantic models and orchestrating the RAG pipeline with performance-optimized execution. The **Health Check Endpoint** (GET /health) returns system status including whether indices are loaded and LLM is enabled. The **Main Query Endpoint** (POST /api/-query) accepts questions with optional course filters and session IDs, processes them through the RAG pipeline, and returns structured responses with answers and sources. The **Streaming Endpoint** (POST /api/query/stream) uses Server-Sent Events (SSE) for real-time token-by-token LLM response generation, reducing perceived latency by delivering incremental results rather than waiting for complete answer generation. Request and response data structures are validated using Pydantic models to ensure type safety and data consistency. The backend leverages FastAPI's asynchronous capabilities to handle concurrent requests efficiently without blocking.

Deployment Architecture. The system is deployed using Docker Compose with two containerized services. The **API Service** runs a Python 3.12 container executing FastAPI with mounted data volumes for indices and metadata, configured via environment variables for LLM provider selection. FAISS indices are preloaded into memory at startup to eliminate loading overhead during query processing, enabling sub-second retrieval latency. The **Frontend Service** operates a Node.js container running Vite development server with environment variable VITE_API_BASE_URL pointing to the API service, ensuring proper backend communication and hot-reload support for development.

3.2.4 Data Flow

This section boast comprehensive operation of the whole system through two primary data flows that work together to enable retrieval-augmented generation. The **Indexing Flow** represents the offline preprocessing pipeline that transforms raw course docu-

ments into searchable vector representations, creating the knowledge base that the system queries. The **Query Flow** represents the runtime pipeline that processes user questions through retrieval, reranking, and generation stages to produce contextual answers. These flows are complementary: the indexing flow prepares the data infrastructure, while the query flow leverages this infrastructure to serve user requests.

3.2.4.1 Indexing Flow

The indexing pipeline processes documents through a sequential workflow designed to transform raw documents into searchable vector representations with performance optimizations throughout.¹ The process begins with Document Ingestion, where raw documents in PDF, DOCX, or PPTX formats are placed in the directory structure `data/raw/<CourseName>/`². The **Document Converter** converts documents to page images at 220 DPI resolution, which are stored in `data/converted/`³ for subsequent OCR processing. The **OCR Engine** utilizes PaddleOCR 3.2 [22], [23] to extract text and structure information, producing JSON files containing text content, bounding boxes, and confidence scores in `data/data/`⁴. The **Extraction Module** performs polygon-based text region extraction from OCR results, utilizing bounding box coordinates to produce annotated outputs (`result.txt`, `result.json`) and debug visualizations. The **Metadata Attacher** enriches extracted text with document metadata including course information, page indices, and structural annotations, ensuring traceability to source documents. The **Syllabus Parser** and **Material Parser** apply specialized parsing for syllabus and material documents to extract structured information such as course metadata, chapter organization, and page indices. The **Text Chunker** segments text into overlapping chunks (default 512 tokens with 50-token overlap) using a sliding window approach with associated metadata, preserving semantic con-

¹The `/data` folder serves as the common storage location for all preprocessing data.

²The `/data/raw` folder contains the original input documents organized by course name, where each course has its own subdirectory.

³The `/data/converted` folder (where `converted` is nested within `data`) contains files converted from PDF to PNG format.

⁴The `/data/data` folder (where `data` is nested within the parent `data` folder) contains data extracted from converted files using OCR [22], [23].

text across boundaries. The **Embedding Model** converts text chunks to compact 384-dimensional vector embeddings using the all-MiniLM-L6-v2 model, balancing semantic quality with computational efficiency and memory footprint, with the **Batch Processor** handling chunks in batches to maximize throughput and optimize computational resources. The **Normalization Component** applies L2-normalization to create unit vectors, enabling cosine similarity computation via computationally efficient inner product operations rather than expensive cosine calculations. Finally, the **Index Manager** creates FAISS indices with IndexFlatIP for exact similarity search, storing normalized embeddings in the **FAISS Index** at `data/indices/index.faiss` while the **Metadata Store** maintains synchronized metadata mappings in `data/indices/metadata.json`⁵ with shared vector IDs, enabling efficient similarity search with sub-500ms retrieval latency for corpora exceeding 10,000 chunks and text reconstruction across the document corpus.

3.2.4.2 Query Flow

The query pipeline processes user questions through a multi-stage retrieval and generation workflow with conversation history integration, optimized for low latency and high throughput. The process initiates with **Query Input**, where users submit natural language questions via the web UI or REST API. The **Query Rewriter** optionally employs an LLM, such as Qwen or Gemini [8], to reformulate queries for improved retrieval quality, fetching previous conversation context from **Conversation History Storage** to understand follow-up questions and resolve pronouns or references. The **Query Embedder** converts the query text to a compact 384-dimensional embedding vector using the lightweight all-MiniLM-L6-v2 model optimized for CPU inference, then L2-normalizes it using $\hat{q} = q/\|q\|_2$ to create a unit vector, ensuring consistency in the vector space. The **Vector Searcher** performs rapid top- k similarity search (default $k = 10$) using the **FAISS Index** [14] with IndexFlatIP through efficient cosine similarity computation via inner product operations, achieving retrieval latency within 500 milliseconds for corpora exceeding 10,000 chunks, then fetches the ac-

⁵The `/data/indices` folder contains `metadata.json` and `index.faiss` files for each course.

tual chunk text and metadata from the **Metadata Store** using the retrieved vector IDs. The **Reranker** applies the FlagEmbedding cross-encoder (BGE Reranker) [19] to compute fine-grained relevance scores through joint query-passage encoding, applying softmax normalization for confidence estimation and returning the top-ranked chunks (typically $k/2$) with highest confidence scores. The **Context Formatter** retrieves conversation history (limited to 10 most recent messages to balance context richness with processing speed) from **Conversation History Storage**, then organizes the reranked chunks with their metadata (course ID, page index, chapter) and incorporates previous queries and responses to enable multi-turn dialogue understanding. The **LLM Answer Generator** leverages a large language model through the LLM Client abstraction (supporting Gemini, Ollama, and Qwen [8]) to generate answers using a prompt template that includes the user’s query and formatted context, producing natural language responses grounded in the retrieved course materials with total answer generation completing within 30 seconds. Finally, the **Response Generator** packages the LLM-generated answer with source citations (course names, page numbers, chunk references), saves the complete query-response pair to **Conversation History Storage** for future interactions, and returns the structured response to the user via the API backend and frontend chat panel.

Chapter 4

Implementation and Testing

*In Chapter 4, we present the detailed implementation of the RAG system and comprehensive testing strategy. The chapter is organized into two main parts: **Implementation** describes the concrete realization of each system module including preprocessing, embedding, indexing, query pipeline, and API components; **Testing** outlines our testing strategy, unit test plans for each module, and establishes the foundation for comprehensive system verification.*

4.1 Overview

This section provides an overview of the system implementation, including the technology stack, project structure, and development workflow.

4.1.1 Technology Stack

The RAG system is implemented using a modern Python-based technology stack that balances performance, maintainability, and ease of deployment.

4.1.1.1 Core Technologies

Programming Language:

- Python 3.10+ - Primary development language
- TypeScript - Frontend development

Document Processing:

- PaddleOCR 3.2 [22], [23] - Optical Character Recognition
- pdf2image - PDF to image conversion
- LibreOffice - Document format conversion
- tiktoken - Token counting and text segmentation

Machine Learning & Embeddings:

- all-MiniLM-L6-v2 [13] - Embedding model (384 dimensions)
- FlagEmbedding [19] - Cross-encoder reranking
- FAISS [14] - Vector similarity search

Backend Infrastructure:

- FastAPI - RESTful API framework
- Pydantic - Data validation and serialization
- Uvicorn - ASGI server
- Python-dotenv - Environment configuration

LLM Providers (one selected at runtime for query rewriting):

- Google Gemini API - Cloud-based generation
- Ollama - Local LLM deployment
- Qwen [8] - Alternative LLM provider (uses Ollama backend)

Note: The system selects one provider via the `MODEL_USING` environment variable. Only the selected provider is used for query rewriting and answer generation.

Frontend:

- React 19 - UI framework

- Vite - Build tool and dev server
- TailwindCSS - Styling framework

Containerization:

- Docker - Container runtime
- Docker Compose - Multi-container orchestration

4.1.2 Project Structure

The project follows a modular architecture with clear separation of concerns:

Table 4.1: Project directory structure (tabular form)

Path	Description
CSE_course_RAG/	Project root
preprocessing/	Document processing modules
img_process/	Image conversion
syllabus/	Syllabus parsing
material/	Material parsing
path_process/	Path utilities
chunking.py	Text chunking
dectector.py	OCR detection
indexing_pipeline.py	Indexing pipeline
models/	ML models and utilities
embedding.py	Embedding generation
indexing.py	FAISS operations
reranker.py	Cross-encoder reranking
debug_utils.py	Debugging tools

Continued on next page

Path	Description
rag/	RAG pipeline
query_pipeline.py	End-to-end retrieval
llm_client.py	LLM abstraction
query_rewriter.py	Query optimization
conversation.py	History management
api/	FastAPI backend
main.py	API endpoints
frontend/	React frontend
src/	Source code
config/	Configuration
config.py	Centralized config
data/	Data storage
raw/	Original documents
converted/	Converted images
processed/	Parsed data
indices/	FAISS indices
run.py	CLI orchestration
dataset.py	Dataset downloader

4.1.3 Development Workflow

The system is designed to be developed and deployed through a series of pipeline stages:

1. **Data Preparation:** Download or prepare course documents in `data/raw/`
2. **Preprocessing:** Run `python run.py --convert --syllabus --material --merge`
3. **Indexing:** Run `python run.py --index` to build FAISS indices

4. **Development:** Start backend and frontend services via Docker Compose
5. **Testing:** Run unit tests and integration tests
6. **Deployment:** Deploy containerized services to production

4.1.4 Configuration Management

The system uses a centralized configuration module (`config/config.py`) that reads from environment variables defined in the `.env` file:

LLM Provider Configuration:

- `MODEL_USING` - LLM provider selection (`gemini/ollama/qwen`)
- `GEMINI_MODEL` - Gemini model variant (e.g., `gemini-2.5-flash`)
- `GEMINI_API_KEY` - API key for Google Gemini service
- `OLLAMA_MODEL` - Ollama model name (e.g., `llama2`)
- `OLLAMA_BASE_URL` - Ollama service endpoint (e.g., `http://ollama:11434`)
- `QWEN_MODEL` - Qwen model variant (e.g., `qwen2.5:3b`)
- `QWEN_BASE_URL` - Qwen service endpoint (e.g., `http://ollama:11434`)

API Configuration:

- `VITE_API_BASE_URL` - Backend API base URL for frontend
(e.g., `http://localhost:8000/api`)

Query Rewriting Configuration:

- `ENABLE_QUERY_REWRITING` - Enable/disable query rewriting feature
(`true/false`)
- `QUERY_REWRITER_TEMPERATURE` - LLM temperature for query rewriting (0.0-1.0, default 0.1)

This approach by centralizing configuration enables easy configuration across different deployment environments without code changes.

Note: The system supports multiple LLM provider configurations (Gemini, Ollama, Qwen) to provide deployment flexibility. While all provider credentials are defined in the environment file, only one provider is active at runtime, selected via the `MODEL_USING` variable. The LLM Client abstraction (implemented in `rag/llm_client.py`) dynamically loads the appropriate provider based on this configuration. This design enables:

- **Easy Provider Switching:** Change between cloud-based (Gemini) and local (Ollama/Qwen) deployments by modifying a single variable. For example, setting `MODEL_USING=gemini` activates the Gemini API with `gemini-2.5-flash`, while `MODEL_USING=qwen` switches to the local Qwen model (`qwen2.5:3b`) running on Ollama at `http://ollama:11434`.
- **Development Flexibility:** Use different providers for development (local Ollama) and production (Gemini API) without reconfiguring credentials. The same `.env` file contains all provider configurations, eliminating the need to manage separate configuration files for different environments.

4.2 Implementation

This section details the concrete implementation of each system module, including algorithms, and code structure.

4.2.1 Preprocessing Module Implementation

The preprocessing module transforms raw course documents into structured, chunked text ready for embedding and indexing. This section shows how all the preprocessing components work together to process course documents. The orchestration layer coordinates the Document Converter, OCR Engine, Extraction Module, Metadata Attacher, Syllabus Parser, Material Parser, and Text Chunker to transform raw documents into structured chunks.

This pipeline orchestrates all preprocessing components in sequence: raw documents are converted to images, OCR extracts text with bounding boxes, the Extraction Module isolates text regions, the Metadata Attacher adds document metadata, parsers extract structured information, and the Text Chunker creates overlapping chunks ready for embedding.

Algorithm 1 Preprocessing Pipeline Orchestration

Input: Raw document files F , course name C , chunk size $s = 512$, overlap $o = 50$

Output: List of all chunks $AllChunks$ ready for embedding

```
1: Initialize empty list:  $AllChunks \leftarrow []$ 
2: for each document file  $f \in F$  do
3:    $images \leftarrow \text{DocumentConverter.convert}(f)$  ▷ Convert to images
4:   for each image  $img \in images$  do
5:      $ocr\_results \leftarrow \text{OCREngine.detect}(img)$  ▷ Extract text with bboxes
6:      $extracted\_text \leftarrow \text{ExtractionModule.extract}(ocr\_results)$  ▷ Polygon
       extraction
7:      $metadata \leftarrow \text{extract\_doc\_metadata}(f, C)$  ▷ Course, page, chapter info
8:      $enriched\_text \leftarrow \text{MetadataAttacher.attach}(extracted\_text, metadata)$ 
9:     if  $f$  is syllabus document then
10:       $parsed\_data \leftarrow \text{SyllabusParser.parse}(enriched\_text)$ 
11:     else if  $f$  is lecture material then
12:       $parsed\_data \leftarrow \text{MaterialParser.parse}(enriched\_text)$ 
13:     end if
14:      $chunks \leftarrow \text{TextChunker.chunk}(parsed\_data, s, o)$  ▷ Token-based
       chunking
15:      $AllChunks \leftarrow AllChunks \cup chunks$ 
16:   end for
17: end for
18: return  $AllChunks$ 
```

4.2.1.0.1 Document Converter The document conversion component converts various document formats to images for OCR processing. The conversion process uses 220 DPI resolution to balance image quality and file size. Higher DPI improves OCR accuracy but increases processing time and storage requirements.

Algorithm 2 PDF to Image Conversion

Input: PDF file path P , output directory D , DPI resolution $r = 220$, format $fmt = \text{"png"}$

Output: List of image paths I

```
1:  $images \leftarrow \text{convert\_from\_path}(P, dpi = r, fmt = fmt)$ 
2: Initialize empty list:  $I \leftarrow []$ 
3: for  $i = 0$  to  $|images| - 1$  do
4:    $output\_path \leftarrow D/f\text{"page\_"}\{i : 03d\}.\{fmt\}$ 
5:   Save  $images[i]$  to  $output\_path$ 
6:   Append  $output\_path$  to  $I$ 
7: end for
8: return  $I$ 
```

4.2.1.0.2 OCR Engine The OCR Engine takes the paths of converted images from the Document Converter as input and performs text detection. It integrates PaddleOCR [22], [23] to extract text with bounding box information from each image. The detector returns structured data including bounding boxes, extracted text, and confidence scores for each detected text region.

Algorithm 3 OCR Text Detection

Input: Image path img , PaddleOCR model M

Output: List of detected text items with bounding boxes and confidence scores

```
1:  $result \leftarrow M.ocr(img, cls = \text{True})$ 
2: Initialize empty list:  $items \leftarrow []$ 
3: for each line  $l \in result[0]$  do
4:    $bbox \leftarrow l[0]$  ▷ Bounding box coordinates
5:    $(text, conf) \leftarrow l[1]$  ▷ Text and confidence
6:   Append  $\{bbox : bbox, text : text, confidence : conf\}$  to  $items$ 
7: end for
8: return  $items$ 
```

4.2.1.0.3 Extraction Module The Extraction Module processes the OCR results (*items*) from the OCR Engine, utilizing the bounding box coordinates to perform polygon-based text region extraction.

Algorithm 4 Polygon-Based Text Extraction

Input: OCR results O with bounding boxes, source image img , output directory D

Output: Extracted text files (result.txt, result.json) and debug visualization

```
1: Initialize empty lists:  $text\_lines \leftarrow []$ ,  $json\_data \leftarrow []$ 
2: for each OCR item  $item \in O$  do
3:    $bbox \leftarrow item.bbox$  ▷ Polygon coordinates
4:    $text \leftarrow item.text$ 
5:    $conf \leftarrow item.confidence$ 
6:   Append  $text$  to  $text\_lines$ 
7:   Append  $\{bbox : bbox, text : text, confidence : conf\}$  to  $json\_data$ 
8: end for
9: Write  $text\_lines$  to  $D/result.txt$ 
10: Write  $json\_data$  to  $D/result.json$ 
11: Draw bounding boxes on  $img$  and save as  $D/debug.png$ 
12: return Paths to output files
```

4.2.1.0.4 Syllabus Parser The Syllabus Parser takes the extracted text and meta-data from the Extraction Module and Metadata Attacher, then extracts structured information specific to syllabus documents. The parser uses pattern matching and heuristics to identify and extract structured information from OCR text.

Algorithm 5 Syllabus Parsing

Input: OCR items O , source file F , page index p , course name C , metadata M

Output: Structured syllabus data S

```
1:  $course\_info \leftarrow \text{extract\_course\_info}(O)$ 
2:  $assessments \leftarrow \text{extract\_assessments}(O)$ 
3:  $description \leftarrow \text{extract\_description}(O)$ 
4:  $S \leftarrow \{$ 
5:    $course\_info : course\_info,$ 
6:    $assessments : assessments,$ 
7:    $course\_des : description,$ 
8:    $metadata : M$ 
9:  $\}$ 
10: return  $S$ 
```

4.2.1.0.5 Material Parser Similar to the Syllabus Parser, the Material Parser takes extracted text and metadata but focuses on processing lecture slides to extract chapter and content information.

Algorithm 6 Material Parsing for Lecture Slides

Input: OCR items O , source file F , page index p , course name C , metadata M

Output: Material data structure Mat

```
1:  $chapter\_num \leftarrow \text{extract\_chapter\_number}(F)$ 
2: Initialize empty string:  $raw\_text \leftarrow ""$ 
3: for each item  $i \in O$  do
4:   if  $i$  contains "text" field then
5:      $raw\_text \leftarrow raw\_text + "" + i.\text{text}$ 
6:   end if
7: end for
8:  $Mat \leftarrow \{$ 
9:    $chapter\_num : chapter\_num,$ 
10:   $page\_index : p,$ 
11:   $source\_file : F,$ 
12:   $raw\_text : raw\_text,$ 
13:   $metadata : M$ 
14:  $\}$ 
15: return  $Mat$ 
```

4.2.1.0.6 Text Chunker The Text Chunker receives the parsed and structured text from the Syllabus Parser or Material Parser and implements token-based text segmentation with overlapping windows to prepare the text for embedding.

The implementation uses the `cl100k_base` encoding (tiktoken) for accurate token counting, ensuring consistent chunk sizes across different text types.

Algorithm 7 Token-Based Chunking with Overlap

Input: Text string T , metadata M , chunk ID prefix $prefix$, chunk size $s = 512$, overlap $o = 50$

Output: List of document chunks $Chunks$

```
1:  $encoding \leftarrow \text{tiktoken.get\_encoding}("cl100k\_base")$ 
2:  $tokens \leftarrow encoding.encode(T)$ 
3: Initialize empty list:  $Chunks \leftarrow []$ 
4:  $chunk\_idx \leftarrow 0$ 
5:  $start \leftarrow 0$ 
6: while  $start < |tokens|$  do
7:    $end \leftarrow start + s$ 
8:    $chunk\_tokens \leftarrow tokens[start : end]$ 
9:    $chunk\_text \leftarrow encoding.decode(chunk\_tokens).strip()$ 
10:  if  $chunk\_text$  is not empty then
11:     $chunk\_id \leftarrow prefix + "-" + \text{format}(chunk\_idx, "04d")$ 
12:    Append  $\{id : chunk\_id, text : chunk\_text, metadata : M\}$  to  $Chunks$ 
13:     $chunk\_idx \leftarrow chunk\_idx + 1$ 
14:  end if
15:   $start \leftarrow end - o$  ▷ Move with overlap
16:  if  $start \geq |tokens|$  then
17:    break
18:  end if
19: end while
20: return  $Chunks$ 
```

4.2.1.0.7 Metadata Attacher Working in parallel with the Extraction Module, the Metadata Attacher enriches the extracted text with document metadata to ensure traceability throughout the pipeline.

This metadata is preserved throughout the pipeline and stored in the Metadata Store alongside vector embeddings, enabling accurate source citation in query responses.

Algorithm 8 Metadata Enrichment

Input: Extracted text T , document metadata M (course_id, page_index, chapter, source_file, doc_type)

Output: Enriched text with metadata $T_{enriched}$

```
1:  $T_{enriched} \leftarrow \{$   
2:    $text : T,$   
3:    $metadata : \{$   
4:      $course\_id : M.course\_id,$   
5:      $page\_index : M.page\_index,$   
6:      $chapter : M.chapter,$   
7:      $source\_file : M.source\_file,$   
8:      $doc\_type : M.doc\_type$   
9:    $\}$   
10:  $\}$   
11: return  $T_{enriched}$ 
```

4.2.2 Embedding and Indexing Implementation

The embedding and indexing module transforms preprocessed text chunks into searchable vector representations. This section shows how the embedding and indexing components work together: the Embedding Model generates dense vector representations, the Batch Processor handles multiple chunks efficiently, the Normalization Component prepares vectors for similarity search, and the Index Manager stores the normalized embeddings in FAISS indices with metadata.

This pipeline orchestrates the embedding and indexing process: preprocessed chunks are embedded in batches using the sentence transformer model, the embeddings are L2-normalized to enable cosine similarity via inner product, metadata is mapped to vector IDs, and the Index Manager creates a FAISS index with synchronized metadata storage.

Algorithm 9 Embedding and Indexing Pipeline Orchestration

Input: List of text chunks $C = \{c_1, c_2, \dots, c_n\}$, course name *course*, batch size $b = 32$

Output: FAISS index I and metadata map M saved to disk

- 1: $model \leftarrow \text{EmbeddingModel.load}(\text{"all-MiniLM-L6-v2"})$ ▷ Load model once
 - 2: Extract text from chunks: $T \leftarrow \{c_i.\text{text} \mid c_i \in C\}$
 - 3: $E \leftarrow \text{BatchProcessor.embed_batch}(T, model, b)$ ▷ Generate embeddings
 - 4: $\hat{E} \leftarrow \text{NormalizationComponent.normalize}(E)$ ▷ L2 normalization
 - 5: $M \leftarrow \text{build_metadata_map}(C)$ ▷ Map chunk IDs to metadata
 - 6: $I \leftarrow \text{IndexManager.create_index}(\hat{E}, M, \text{course})$ ▷ Create and save FAISS index
- return** I, M
-

4.2.2.0.1 Embedding Model The embedding model is loaded once at system startup and cached in memory for reuse across all embedding operations. This initialization process loads the pre-trained all-MiniLM-L6-v2 model from the SentenceTransformers library, which produces 384-dimensional embeddings balancing quality and computational efficiency.

Algorithm 10 Embedding Model Initialization

Input: Model name $M_{name} = \text{"all-MiniLM-L6-v2"}$

Output: Loaded embedding model M , embedding dimension $d = 384$

- 1: $M \leftarrow \text{SentenceTransformer}(M_{name})$
 - 2: $d \leftarrow 384$ ▷ Embedding dimension **return** M, d
-

4.2.2.0.2 Batch Processor Once the model is initialized, the Batch Processor handles multiple text chunks efficiently by leveraging vectorized operations. Batch processing significantly improves throughput compared to processing chunks individually.

Algorithm 11 Batch Embedding Generation

Input: List of text chunks $C = \{c_1, c_2, \dots, c_n\}$, batch size b , embedding model M

Output: Array of embeddings E where each $e_i \in \mathbb{R}^{384}$

- 1: Initialize empty list $E \leftarrow []$
 - 2: Extract text from chunks: $T \leftarrow \{c_i.\text{text} \mid c_i \in C\}$
 - 3: **for** $i = 0$ to $\lceil |T|/b \rceil - 1$ **do**
 - 4: $B \leftarrow T[i \cdot b : (i + 1) \cdot b]$ ▷ Get batch of texts
 - 5: $E_b \leftarrow M.\text{embed_batch}(B)$ ▷ Generate embeddings for batch
 - 6: $E \leftarrow E \cup E_b$ ▷ Append batch embeddings
 - 7: **end for**
 - 8: Convert E to numpy array with dtype float32
 - 9: **return** E
-

4.2.2.0.3 Normalization Component After batch embedding generation, the Normalization Component prepares embeddings for efficient similarity search. Embeddings are L2-normalized to enable cosine similarity computation via inner product.

Algorithm 12 L2 Normalization

Input: Embedding matrix $E \in \mathbb{R}^{n \times 384}$

Output: Normalized embedding matrix $\hat{E}$ where $\|\hat{e}_i\|_2 = 1$ for all rows

```
1: Compute L2 norms:  $N \leftarrow \|E\|_2$  for each row (axis=1)
2: for each norm  $n_i \in N$  do
3:   if  $n_i = 0$  then
4:      $n_i \leftarrow 1$  ▷ Avoid division by zero
5:   end if
6: end for
7: Normalize:  $\hat{E} \leftarrow E/N$  ▷ Element-wise division return  $\hat{E}$ 
```

After normalization, the cosine similarity between vectors u and v becomes:

$$\text{cosine}(u, v) = \frac{u \cdot v}{\|u\| \|v\|} = u \cdot v$$

when both are unit vectors ($\|u\| = \|v\| = 1$).

4.2.2.0.4 Index Manager The Index Manager takes the normalized embeddings from the Normalization Component and orchestrates the complete index lifecycle. Its responsibilities include: (1) creating new FAISS indices from embeddings, (2) loading existing indices from disk, (3) persisting indices and metadata to disk, and (4) managing separate indices for multiple courses. The implementation stores each course's index in `data/indices/<course_name>/` directories, with `index.faiss` (vector index) and `metadata.json` (chunk text and metadata).

Index Creation The Index Manager first creates a new FAISS index from normalized embeddings. We use `IndexFlatIP` (Inner Product) for exact similarity search on normalized vectors, ensuring that cosine similarity can be computed as an inner product.

Algorithm 13 FAISS Index Creation

Input: Normalized embeddings $\hat{E} \in \mathbb{R}^{n \times 384}$, metadata map M , embedding dimension $d = 384$

Output: FAISS index I ready for search

- 1: Create FAISS index: $I \leftarrow \text{IndexFlatIP}(d)$ ▷ Inner Product index
 - 2: Convert $\hat{E}$ to float32 format
 - 3: Add normalized embeddings to index: $I.\text{add}(\hat{E})$ **return** I
-

Index Persistence After creating the index, the Index Manager saves both the FAISS index and metadata to disk for future reuse. This ensures synchronized storage of vectors and their associated metadata.

Algorithm 14 Save Index to Disk

Input: FAISS index I , output path P , metadata map M

Output: Index and metadata saved to disk

- 1: Create directory: $\text{mkdir}(P)$
 - 2: $\text{index_path} \leftarrow P / \text{"index.faiss"}$
 - 3: $\text{metadata_path} \leftarrow P / \text{"metadata.json"}$
 - 4: Save FAISS index: $\text{faiss.write_index}(I, \text{index_path})$
 - 5: Save metadata as JSON: $\text{json.dump}(M, \text{metadata_path})$
-

Index Loading When the system starts or when a query is made, the Index Manager loads existing indices from disk to avoid recomputing embeddings.

Algorithm 15 Load Index from Disk

Input: Input path P

Output: Loaded FAISS index I and metadata map M

- 1: $\text{index_path} \leftarrow P / \text{"index.faiss"}$
 - 2: $\text{metadata_path} \leftarrow P / \text{"metadata.json"}$
 - 3: Load FAISS index: $I \leftarrow \text{faiss.read_index}(\text{index_path})$
 - 4: Load metadata: $M \leftarrow \text{json.load}(\text{metadata_path})$ **return** I, M
-

4.2.2.0.5 FAISS Index Search The FAISS Index provides the low-level search operation that is called by the Vector Searcher component in the query pipeline (Section 4.2.3). The operation performs efficient top- k similarity search on a single index. This low-level operation is called by the Vector Searcher for each course index during multi-index search orchestration.

Algorithm 16 FAISS Index Search Operation

Input: FAISS index I , query embedding $q \in \mathbb{R}^{384}$, number of results k , metadata map

M

Output: List of results R containing $(chunk_id, score, metadata)$ tuples

```

1: Compute query norm:  $n_q \leftarrow \|q\|_2$ 
2: if  $n_q = 0$  then
3:    $n_q \leftarrow 1$  ▷ Avoid division by zero
4: end if
5: Normalize query:  $\hat{q} \leftarrow q/n_q$ 
6: Convert  $\hat{q}$  to float32 and reshape to  $(1, 384)$ 
7: Perform FAISS search:  $(D, Idx) \leftarrow I.search(\hat{q}, k)$ 
8: Initialize empty list:  $R \leftarrow []$ 
9: for each  $(score, idx) \in (D[0], Idx[0])$  do
10:    $chunk\_id \leftarrow M[idx][\"id\"]$ 
11:    $metadata \leftarrow M[idx]$ 
12:   Append  $(chunk\_id, score, metadata)$  to  $R$ 
13: end for
14: return  $R$ 

```

4.2.2.0.6 Metadata Management Metadata is stored separately in JSON format, mapping vector indices to document information. This metadata is created in parallel with embedding generation and synchronized with the FAISS index.

Algorithm 17 Metadata Map Construction

Input: List of document chunks $C = \{c_1, c_2, \dots, c_n\}$

Output: Metadata map M mapping chunk IDs to metadata

```
1: Initialize empty map:  $M \leftarrow \{\}$ 
2: for each chunk  $c_i \in C$  do
3:    $M[c_i.id] \leftarrow \{$ 
4:     "id" :  $c_i.id$ ,
5:     "text" :  $c_i.text$ ,
6:     "metadata" : {
7:       "doc_type" :  $c_i.metadata.doc\_type$ ,
8:       "course_id" :  $c_i.metadata.course\_id$ ,
9:       "source_file" :  $c_i.metadata.source\_file$ ,
10:      "page_index" :  $c_i.metadata.page\_index$ 
11:    }
12:  }
13: end for
14: return  $M$ 
```

4.2.3 Query Pipeline Implementation

The query pipeline orchestrates the end-to-end retrieval and generation process. This section shows how all query processing components work together: the Query Rewriter reformulates queries for better retrieval, the Query Embedder converts queries to vectors, the Vector Searcher retrieves relevant chunks from FAISS indices, the Reranker refines results using cross-encoder scoring, the Context Formatter prepares retrieved content, the Conversation Manager maintains dialogue history, and the LLM Answer Generator produces final responses.

This pipeline orchestrates all query processing components in sequence: queries are optionally rewritten for better retrieval, embedded into vectors, searched against FAISS indices, reranked using cross-encoder models, formatted with conversation history, and finally processed by the LLM to generate contextual answers.

Algorithm 18 Complete Query Pipeline Orchestration

Input: User query q , optional course filter c , session ID s , retrieval count $k_r = 8$, rerank count $k_{rr} = 5$

Output: Answer response with sources and conversation history

```
1:  $q' \leftarrow \text{QueryRewriter.rewrite}(q, c)$  ▷ Optional query reformulation
2:  $q_{emb} \leftarrow \text{QueryEmbedder.embed}(q')$  ▷ Convert to vector
3:  $retrieved \leftarrow \text{VectorSearcher.search}(q_{emb}, c, k_r)$  ▷ FAISS search
4: if  $retrieved$  is empty then return {status : "no_info"}
5: end if
6:  $reranked \leftarrow \text{Reranker.rerank}(q', retrieved, k_{rr})$  ▷ Cross-encoder scoring
7: if  $reranked[0].confidence < \theta$  then return {status : "low_confidence"}
8: end if
9:  $history \leftarrow \text{ConversationManager.get\_history}(s)$  ▷ Get conversation context
10:  $context \leftarrow \text{ContextFormatter.format}(reranked, history)$  ▷ Prepare prompt
11:  $answer \leftarrow \text{LLMAnswerGenerator.generate}(q, context)$  ▷ Generate response
12:  $\text{ConversationManager.save}(s, q, answer)$  ▷ Update history
13: return {status : "ok", answer :  $answer$ , sources :  $reranked$ }
```

4.2.3.0.1 Query Rewriter The Query Rewriter is the first component in the pipeline, taking the raw user query and reformulating it for better retrieval. This optional component uses an LLM to expand or clarify queries, improving retrieval accuracy.

Algorithm 19 LLM-Based Query Rewriting

Input: Original query q , optional course c , LLM client L , temperature $T = 0.3$

Output: Rewritten query q'

```
1:  $prompt \leftarrow \text{build\_rewrite\_prompt}(q, c)$ 
2:  $system \leftarrow \text{"You are a query rewriter..."}$ 
3: try
4:    $q' \leftarrow L.\text{generate}(prompt, system, T, max\_tokens = 100)$ 
5:    $q' \leftarrow q'.strip()$ 
6:   if  $q'$  is empty then
7:      $q' \leftarrow q$  ▷ Fallback to original
8:   end if
9:   catch Exception
10:     $q' \leftarrow q$  ▷ Fallback to original
11:  end catch
12: end try
13: return  $q'$ 
```

4.2.3.0.2 Query Embedder After query rewriting (or using the original query if rewriting is disabled), the Query Embedder converts the text query into a 384-dimensional vector representation using the same embedding model used for document chunks.

Algorithm 20 Query Embedding

Input: Query string q' , embedding model M

Output: Query embedding $q_{emb} \in \mathbb{R}^{384}$

- 1: $q_{emb} \leftarrow M.encode(q')$
 - 2: Normalize: $q_{emb} \leftarrow q_{emb} / \|q_{emb}\|_2$
 - 3: Convert to float32 numpy array
 - 4: **return** q_{emb}
-

4.2.3.0.3 Vector Searcher The Vector Searcher takes the query embedding from the Query Embedder and orchestrates search across multiple course indices. For each relevant index, it calls the FAISS similarity search operation (detailed in Section 4.2.2) and aggregates results from all courses.

This algorithm focuses on multi-course orchestration: it determines which indices to search based on the course filter, calls the low-level FAISS search for each index, and aggregates results across all searched courses.

Algorithm 21 Multi-Index Vector Search Orchestration

Input: Query embedding q_{emb} , optional course filter c , retrieval count k_r , course indices $indices$

Output: List of retrieved chunks R aggregated from all relevant courses

```
1: Initialize empty list:  $R \leftarrow []$ 
2: if course  $c$  is specified and  $c \in indices$  then
3:    $course\_indices \leftarrow \{indices[c]\}$  ▷ Single course
4: else
5:    $course\_indices \leftarrow$  all indices ▷ All courses
6: end if
7: for each course index  $idx \in course\_indices$  do
8:    $(scores, ids) \leftarrow \text{FAISSIndex.search}(idx.index, q_{emb}, k_r)$  ▷ See Sec. 4.2.2
9:   for each  $(score, id) \in (scores, ids)$  do
10:     $chunk\_id \leftarrow idx.chunk\_ids[id]$ 
11:     $metadata \leftarrow idx.metadata[chunk\_id]$ 
12:    Append  $(chunk\_id, metadata.text, score, idx.course)$  to  $R$ 
13:   end for
14: end for
15: Sort  $R$  by score (descending)
16: return  $R$ 
```

4.2.3.0.4 Reranker The Reranker takes the retrieved chunks from the Vector Searcher and applies fine-grained relevance scoring using a cross-encoder model. This two-stage approach (FAISS retrieval + reranking) improves precision by capturing query-passage interactions that embedding similarity alone cannot.

Algorithm 22 Cross-Encoder Reranking with Softmax Normalization

Input: Query string q , list of retrieved chunks C , reranker model R , top- k value k

Output: Ranked list of top- k chunks with confidence scores

```
1: Create query–passage pairs:  $P \leftarrow \{(q, c_i.\text{text}) \mid c_i \in C\}$ 
2: Compute raw relevance scores:  $S \leftarrow \{R.\text{compute\_score}(p) \mid p \in P\}$ 
3:                                      $\triangleright$  — Begin Softmax Normalization —
4: Find maximum score:  $s_{\max} \leftarrow \max(S)$                                       $\triangleright$  For numerical stability
5: Initialize confidence list:  $Conf \leftarrow []$ 
6: for each score  $s_i \in S$  do
7:    $exp\_score \leftarrow \exp(s_i - s_{\max})$                                       $\triangleright$  Softmax numerator:  $\exp(s_i - s_{\max})$ 
8:   Append  $exp\_score$  to  $Conf$ 
9: end for
10: Compute normalizer:  $Z \leftarrow \sum(Conf)$   $\triangleright$  Softmax denominator:  $\sum_j \exp(s_j - s_{\max})$ 
11: if  $Z = 0$  then
12:    $Z \leftarrow 1.0$                                       $\triangleright$  Avoid division by zero
13: end if
14: Normalize confidences:  $Conf \leftarrow \{c_i/Z \mid c_i \in Conf\}$   $\triangleright$  Softmax:  $\frac{\exp(s_i - s_{\max})}{\sum_j \exp(s_j - s_{\max})}$ 
15:                                      $\triangleright$  — End Softmax Normalization —
16: Create tuples:  $Ranked \leftarrow \{(c_i, Conf[i]) \mid c_i \in C\}$ 
17: Sort by confidence (descending)
18: return First  $k$  elements of  $Ranked$ 
```

4.2.3.0.5 Conversation Manager The Conversation Manager maintains dialogue history across multiple turns, enabling context-aware responses. It stores previous user queries and assistant answers, formatting them for inclusion in the LLM prompt.

Algorithm 23 Conversation History Management

Input: Session ID s , maximum messages $m = 10$

Output: Formatted history string H or None

```
1: if session  $s$  not in sessions then return None
2: end if
3:  $messages \leftarrow$  last  $m$  messages from session  $s$ 
4: Initialize empty list:  $formatted \leftarrow []$ 
5: for each message  $msg \in messages$  do
6:   Append " $msg.role: msg.content$ " to  $formatted$ 
7: end for
8:  $H \leftarrow \text{join}(formatted, "\n")$ 
9: return  $H$ 
```

4.2.3.0.6 Context Formatter The Context Formatter takes the reranked chunks and conversation history from previous components and prepares a structured prompt for the LLM. It combines retrieved context with dialogue history to enable contextual answer generation.

Algorithm 24 Context Formatting for LLM

Input: Reranked chunks R , conversation history H , top- k value $k = 5$

Output: Formatted context string C

```
1: Extract top  $k$  chunks:  $top\_chunks \leftarrow R[0 : k]$ 
2: Initialize context:  $C \leftarrow \text{"Retrieved Context:\n"}$ 
3: for each chunk  $c \in top\_chunks$  do
4:   Append " $c.text$ \n\n" to  $C$ 
5: end for
6: if  $H$  is not None then
7:   Append "\nConversation History:\n $H$ " to  $C$ 
8: end if
9: return  $C$ 
```

4.2.3.0.7 LLM Answer Generator The LLM Answer Generator is the final component, taking the formatted context and conversation history to generate a natural language answer. It provides a unified interface for multiple LLM providers (Gemini, Ollama, Qwen).

Algorithm 25 LLM-Based Answer Generation

Input: User query q , formatted context C , LLM client L , temperature $T = 0.7$

Output: Generated answer a

```
1:  $system \leftarrow$  "You are a helpful assistant..."
2:  $prompt \leftarrow$  "Question:  $q$ \n\nContext:  $C$ \n\nAnswer:"
3: try
4:    $a \leftarrow L.generate(prompt, system, T, max\_tokens = 500)$ 
5:    $a \leftarrow a.strip()$ 
6:   catch Exception  $e$ 
7:      $a \leftarrow$  "Unable to generate answer. Error: " +  $e$ 
8:   end catch
9: end try
10: return  $a$ 
```

4.2.3.0.8 Response Generator The Response Generator is the final component in the query pipeline. It takes the LLM-generated answer from the LLM Answer Generator and packages it with source citations (including course names, page numbers, and chunk references) into a structured response format. It then saves the complete query-response pair to the Conversation History Storage before returning the response to the user via the API backend, completing the query-answer cycle.

This component ensures that users receive not only the generated answer but also the source citations for transparency and verification, while maintaining conversation context for multi-turn dialogues.

Algorithm 26 Response Generation and Packaging

Input: User query q , LLM answer a , reranked chunks R , session ID s , top- k sources

$$k_s = 3$$

Output: Structured response $resp$ with answer and citations

```
1: Initialize source list:  $sources \leftarrow []$ 
2: for each chunk  $r \in R[0 : k_s]$  do
3:    $citation \leftarrow \{$ 
4:      $course : r.metadata.course\_id,$ 
5:      $page : r.metadata.page\_index,$ 
6:      $chunk\_id : r.chunk\_id,$ 
7:      $text : r.text,$ 
8:      $confidence : r.confidence$ 
9:    $\}$ 
10:   Append  $citation$  to  $sources$ 
11: end for
12:  $resp \leftarrow \{$ 
13:    $status : "ok",$ 
14:    $question : q,$ 
15:    $answer : a,$ 
16:    $sources : sources,$ 
17:    $llm\_used : true$ 
18:  $\}$ 
19: ConversationHistory.save( $s, q, a$ ) ▷ Save to conversation storage
20: return  $resp$ 
```

4.2.4 API and Frontend Implementation

The API and Frontend layer exposes the RAG system to end users through a FastAPI backend and React frontend. For the overall architecture and design, see Section 3.2.3 in the Methodology chapter. This section provides minimal implementation details for the three main components: React Frontend, FastAPI Backend, and Deployment Architecture.

4.2.4.0.1 React Frontend The React frontend implements the chat interface using component-based architecture in `frontend/src/`:

```
1 function ChatInterface() {  
2   const [messages, setMessages] = useState([]);  
3   const handleSubmit = async (input) => {  
4     const response = await queryRAG(input, course, sessionId);  
5     setMessages([...messages,  
6       { role: 'user', content: input },  
7       { role: 'assistant', content: response.answer,  
8         sources: response.sources }  
9     ]);  
10  };  
11 }
```

Listing 4.1: Chat Interface Component

4.2.4.0.2 FastAPI Backend The backend implements three endpoints in `api/main.py`.

The **health check endpoint** verifies system operational status, allowing clients to check whether indices are loaded and LLM is enabled before sending queries:

Listing 4.2: Health Check Endpoint

```
1 @app.get("/health")  
2 async def health():
```

```

3     return {"status": "ok", "indices_loaded": True,
4            "llm_enabled": llm_client.enabled}

```

The **main query endpoint** processes user questions through the RAG pipeline, invoking retrieval, reranking, and LLM generation to produce answers with source citations:

Listing 4.3: Query Endpoint

```

1 @app.post("/api/query")
2 async def query(request: QueryRequest):
3     result = pipeline.answer(request.question, request.course)
4     if llm_client.enabled:
5         answer = llm_client.generate_answer(
6             request.question,
7             extract_contexts(result["reranked"]),
8             conversation_manager.format_history(session_id)
9         )
10    return QueryResponse(status="ok", answer=answer,
11                        sources=build_sources(result["reranked"]))

```

The **streaming endpoint** provides real-time response generation using Server-Sent Events, enabling the frontend to display LLM tokens in time as they are generated for improved user experience:

Listing 4.4: Streaming Endpoint

```

1 @app.post("/api/query/stream")
2 async def stream_query(request: QueryRequest):
3     async def generate():
4         for token in llm_client.stream_generate(...):
5             yield f"data: {json.dumps({'token': token})}\n\n"
6     return StreamingResponse(generate(), media_type="text/event-stream")

```

The FastAPI backend uses **Pydantic models** for request/response validation:

Listing 4.5: Pydantic Models

```
1 class QueryRequest(BaseModel):
2     question: str
3     course: Optional[str] = None
4     session_id: Optional[str] = None
5
6 class QueryResponse(BaseModel):
7     status: str
8     answer: str
9     sources: List[SourceChunk]
```

4.2.4.0.3 Deployment Architecture The system is deployed using Docker Compose with two containerized services that work together. The **API Service** runs a Python 3.12 container executing FastAPI on port 8000, with the `./data` directory mounted to `/app/data` for accessing FAISS indices and metadata files. Environment variables are loaded from `.env` file to configure LLM providers (Gemini, Ollama, Qwen) and API keys. The **Frontend Service** runs a Node.js container with Vite development server on port 5173, configured with `VITE_API_BASE_URL` environment variable pointing to the API service for backend communication.

This containerized architecture ensures consistent deployment across development and production environments, with automatic service orchestration and hot-reload support for development workflows.

```
1 services:
2   api:
3     build: ./api
4     ports: ["8000:8000"]
5     volumes: [".:/data:/app/data"]
6     env_file: .env
7
```



```
8 frontend:
9   build: ./frontend
10  ports: ["5173:5173"]
11  environment:
12    VITE_API_BASE_URL: http://localhost:8000
13  depends_on: [api]
```

Listing 4.6: Docker Compose Services

4.3 Testing

The testing strategy ensures system reliability, correctness, and performance through comprehensive test coverage across multiple levels.

4.3.1 Testing Strategy

The testing approach follows the testing pyramid methodology, allocating test effort across three levels: **Unit Tests** (70%) verify individual functions and classes in isolation, **Integration Tests** (20%) validate interactions between components, and **End-to-End Tests** (10%) simulate complete user workflows. This distribution ensures thorough validation of core functionality while maintaining manageable test execution time. Tests are organized using pytest framework with coverage reporting enabled, allowing continuous monitoring of code quality.

4.3.1.1 Unit Testing

Unit tests validate the correctness of individual modules in isolation, focusing on the core RAG components.

The **Embedding Module** tests verify model initialization, single text embedding generation, batch processing capabilities, empty text handling, and embedding consistency. These tests ensure that the sentence transformer model produces valid 384-dimensional vectors and maintains deterministic behavior.

The **Indexing Module** tests cover FAISS index creation, document addition, similarity search operations, index persistence (save/load), and metadata mapping. These validate that vector storage and retrieval operations function correctly across different scenarios.

The **Reranker Module** tests examine cross-encoder initialization, relevance scoring, softmax normalization for confidence estimation, and proper handling of edge cases such as empty or single-passage inputs.

The **Query Pipeline** tests verify component initialization, presence of required de-

dependencies (embedding, indices, reranker), and query input validation.

4.3.1.2 Integration Testing

Integration tests verify that components work correctly together, particularly focusing on the retrieval pipeline where embedding, FAISS search, and metadata retrieval must coordinate seamlessly. Tests validate the complete retrieval and reranking flow, ensuring that query embeddings are generated, similarity search returns relevant chunks, and the reranker properly scores results. Course filtering tests confirm that the system correctly restricts search to specific course indices when requested. Query rewriting integration tests verify that LLM-based query reformulation improves retrieval quality by expanding or clarifying user queries. Edge case handling tests ensure graceful degradation for empty queries, unknown courses, and low-confidence results.

4.3.1.3 End-to-End Testing

End-to-end tests simulate real user interactions through the API layer, validating the complete workflow from HTTP request to JSON response. Tests cover the health check endpoint to verify system operational status, the main query endpoint for complete question-answering workflows, and the streaming endpoint for real-time token generation. Conversation context tests ensure that multi-turn dialogues maintain proper context across sequential queries. Input validation tests confirm that the API properly rejects malformed requests with appropriate error codes (422 for validation errors). The new conversation flag test verifies that users can reset conversation history when needed.

4.3.2 Test Execution Results

The test suite was executed successfully with all tests passing, demonstrating comprehensive validation of system functionality across all testing levels.

4.3.2.1 Test Summary

Table 4.2: Test execution results by category

Test Category	Tests	Passed	Status
<i>Unit Tests</i>			
Embedding	5	5	100%
Indexing	5	5	100%
Reranker	6	6	100%
Query Pipeline	3	3	100%
Unit Tests Subtotal	19	19	100%
<i>Integration Tests</i>			
Retrieval Pipeline	6	6	100%
Integration Tests Subtotal	6	6	100%
<i>End-to-End Tests</i>			
API Endpoints	8	8	100%
E2E Tests Subtotal	8	8	100%
Total	33	33	100%

4.3.2.1.1 Unit Test Results Analysis The perfect pass rate (19/19) reflects the maturity and reliability of the core RAG components. Each module demonstrates consistent behavior across diverse test scenarios, from basic functionality to edge cases. The embedding module’s 100% success rate validates that the sentence transformer model correctly produces 384-dimensional vectors with deterministic behavior. The indexing module’s perfect score confirms that FAISS operations (creation, addition, search, persistence) function reliably across different data scenarios. The reranker’s flawless performance demonstrates robust cross-encoder scoring with proper softmax normalization, even handling edge cases like empty or single-passage inputs. The query pipeline tests confirm that component initialization and orchestration work correctly,

with proper dependency management and input validation.

4.3.2.1.2 Integration Test Results Analysis Integration tests (6/6 passed) validate the system's ability to orchestrate multiple components into coherent workflows. The successful execution confirms that data flows correctly between modules with proper error handling and fallback mechanisms. The retrieval pipeline integration demonstrates seamless coordination between embedding generation, FAISS vector search, and metadata retrieval. Course filtering tests verify that the system correctly restricts search scope when requested, maintaining data isolation between different course indices. Query rewriting integration tests show that LLM-based reformulation successfully improves retrieval quality by expanding ambiguous queries. The pipeline maintains consistency across different query types and course configurations, demonstrating robust integration between all stages.

4.3.2.1.3 End-to-End Test Results Analysis E2E tests (8/8 passed) demonstrate that the API layer correctly integrates with the underlying RAG pipeline, providing a reliable interface for user interactions. All endpoint tests pass, confirming that request validation using Pydantic models works correctly, rejecting malformed inputs with appropriate HTTP 422 status codes. The health check endpoint validates system operational status and LLM availability. The main query endpoint successfully orchestrates the complete RAG workflow from HTTP request to JSON response, including retrieval, reranking, and LLM generation. Conversation context tests validate that multi-turn dialogues maintain proper history across sequential queries using session IDs, enabling coherent follow-up questions. The streaming endpoint tests confirm that Server-Sent Events work correctly for real-time token generation, providing responsive user experiences without blocking. The new conversation flag test verifies that users can reset conversation history when needed.

4.3.2.2 Coverage Analysis

Table 4.3: Code coverage by module

Module	Coverage	Status
<i>Core RAG Modules</i>		
models/reranker.py	97%	Excellent
preprocessing/manifest.py	100%	Excellent
rag/query_pipeline.py	90%	Excellent
rag/conversation.py	91%	Excellent
models/load_model.py	86%	Good
models/embedding.py	79%	Good
<i>API and Supporting Modules</i>		
api/main.py	84%	Excellent
config/config.py	79%	Good
rag/query_rewriter.py	72%	Good
rag/llm_client.py	62%	Good
models/indexing.py	57%	Moderate
rag/llm_provider.py	33%	Moderate
Overall Project Coverage	50%	(1,321 statements)

The coverage metrics reflect the combined execution of all 33 tests (19 unit + 6 integration + 8 E2E). E2E tests significantly boost coverage by exercising the API layer and supporting infrastructure. The API main module achieves 84% coverage through comprehensive endpoint testing (health check, query, streaming, conversation management). The conversation module reaches 91% coverage by validating session management and history tracking across multi-turn dialogues. Configuration module coverage increases to 79% through environment variable access during API initialization. LLM client coverage improves to 62% through E2E tests exercising

provider initialization and response generation. The query pipeline maintains 90% coverage through all three testing levels. The reranker (97%) and manifest (100%) modules demonstrate excellent coverage through focused unit testing. The overall project coverage of 50% across 1,321 statements reflects comprehensive testing of the complete RAG system from core components through API endpoints, validating both individual module correctness and end-to-end system integration.

Chapter 5

Experiments

*In Chapter 5, it presents experimental evaluation of our RAG-based question-answering system for CSE course materials. The chapter is organized into three main sections: **Experimental Setup** describes the dataset composition, two baseline configurations, query type categorization, and implementation specifications; **Evaluation Methodology** outlines the evaluation framework including metrics for answer generation quality, retrieval performance, system performance, conversation context memory, and analysis of query rewriting impact; **Experimental Results** presents comprehensive results comparing the two configurations across all evaluation dimensions, demonstrating the impact of query rewriting on answer quality, retrieval effectiveness, system performance, and conversation context utilization.*

5.1 Experimental Setup

This section describes the experimental setup for evaluating the RAG-based question-answering system, including dataset details, system configurations, and implementation specifics.

5.1.1 Dataset

Throughout all experiments, we use a dataset collected from multiple sources including HCMUT LMS [20] and Coursewave [21]. The dataset encompasses multiple CSE courses, each containing syllabus documents and lecture materials (slides), enabling comprehensive evaluation across different course domains and document types. Table 5.1 provides an overview of the dataset composition.

Table 5.1: Dataset Statistics

Statistic	Value
Number of Courses	32
Total Documents	183
Syllabus Documents	153
Material Documents (Slides)	30
Total Text Chunks	15,013
Average Chunks per Course	469.2
Total Tokens (approximate)	6,140,317

5.1.2 System Configurations

To evaluate the effectiveness of different retrieval and query optimization strategies, we examine two baseline configurations that enable systematic analysis of query rewriting impact. This experimental design addresses several critical evaluation objectives: first, comparing a standard RAG pipeline (FAISS + Reranker) against an enhanced version with query rewriting (FAISS + Query Rewriter + Reranker) isolates the specific contribution of query optimization, allowing us to quantify whether LLM-based query rewriting improves retrieval quality and answer generation; second, the two-baseline approach enables comprehensive analysis across multiple dimensions including answer quality (Query-Answer Similarity, Answer Faithfulness), retrieval performance (Query-Chunk Similarity, Top Chunk Score), and system performance (latency, throughput), providing insights into quality-performance trade-offs; third, this configuration enables query type analysis to identify which query characteristics (short ambiguous, course information, specific, full questions) ben-

enefit most from rewriting versus those that do not, supporting selective application of query optimization strategies; and fourth, maintaining identical retrieval architecture (FAISS + Reranker) across both configurations ensures that observed differences are attributable to query rewriting rather than other pipeline variations, enabling fair comparison and clear interpretation of results. The two baseline configurations are:

RAG Baseline (FAISS + Reranker). This configuration implements the standard retrieval-augmented generation [11] pipeline with two-stage retrieval. The system first retrieves candidate chunks using FAISS [14] similarity search based on semantic embeddings, then applies cross-encoder reranking [18], [19] to refine the results by considering query-document interactions. The reranked chunks are subsequently passed to the LLM [1] for answer generation. This represents the main baseline for comparison, demonstrating the effectiveness of combining efficient vector search with precision-focused reranking.

RAG + Query Rewriter. This configuration extends the RAG baseline by incorporating query rewriting as an additional optimization step. Before retrieval, the original user query is rewritten by an LLM [1] to improve semantic matching with document content. The rewritten query is then used for both FAISS retrieval [14] and cross-encoder reranking [19], following the same two-stage retrieval pipeline as the baseline. This configuration tests whether query optimization through LLM-based rewriting improves retrieval quality and subsequent answer generation compared to the standard RAG baseline, particularly for ambiguous or conversational queries.

5.1.3 Query Type Categorization

To evaluate system performance across different scenarios, the evaluation dataset includes diverse query types:

- **Short Ambiguous Queries:** Brief queries with limited context (e.g., "prerequisites", "grading policy") that require disambiguation and expansion to match relevant document content.
- **Course Information Queries:** Queries seeking course-specific information (e.g.,

"course objectives", "instructor contact") that may benefit from contextual expansion.

- **Specific Queries:** Well-formed queries targeting specific information (e.g., "What is the deadline for project submission?") that are already clear and may not require significant rewriting.
- **Full Questions:** Complete questions with full context (e.g., "What are the prerequisites for the Introduction to Computing course?") that are well-formed and may show minimal improvement from rewriting.

This categorization enables analysis of how query rewriting effectiveness varies across different query characteristics, as presented in the results section.

5.1.4 Implementation Details

The experimental evaluation is conducted using the following implementation specifications:

- **Embedding Model:** Sentence-transformer model for generating semantic embeddings of queries and documents, ensuring consistent semantic representation across the retrieval and evaluation pipeline.
- **Vector Database:** FAISS [14] for efficient similarity search over document embeddings.
- **Reranker:** Cross-encoder reranker [19] for refining retrieval results by considering query-document interactions.
- **LLM:** Large language model [1] for both query rewriting and answer generation.
- **Retrieval Parameters:** Top- k chunks are retrieved and reranked for each query, with k selected to balance retrieval coverage and computational efficiency.

All experiments are conducted on the same dataset and evaluation queries to ensure fair comparison between configurations. The evaluation metrics are computed consistently across both configurations to enable comparative analysis.

5.2 Evaluation Methodology

This section describes the evaluation methodology and metrics used to assess the RAG system’s performance across different configurations. The evaluation framework encompasses quantitative metrics for answer quality, retrieval performance, system performance, conversation context memory, and analysis of query rewriting impact to provide comprehensive insights into system behavior.

5.2.1 Answer Generation Quality

Answer generation quality is evaluated using semantic similarity metrics that do not require ground truth reference answers, making them suitable for datasets extracted from raw course materials. Two primary metrics are employed:

Query-Answer Semantic Similarity quantifies the semantic alignment between the original user query and the generated answer by computing the cosine similarity between their respective embeddings. The embeddings are generated using the same sentence-transformer model employed for document indexing, ensuring consistent semantic representation across the retrieval and evaluation pipeline. This metric serves as a proxy for answer relevance when reference answers are unavailable.

Answer Faithfulness measures the extent to which generated answers are grounded in the retrieved context. It is computed as the average cosine similarity between the answer embedding and the embeddings of the top- k retrieved chunks. This metric is critical for detecting hallucination and ensuring that answers are derived from the retrieved course materials rather than the language model’s parametric knowledge.

Both metrics are computed for each configuration (baseline and query rewriting variant) to enable comparative analysis of answer quality improvements.

5.2.2 Retrieval Performance

Retrieval performance is assessed using metrics that evaluate the relevance of retrieved chunks without requiring ground truth relevance labels:

Query-Chunk Semantic Similarity measures the average cosine similarity between the original query embedding and the embeddings of the top- k retrieved chunks. This metric quantifies how well the retrieved chunks match the query intent, using the same embedding model as the indexing process to ensure semantic consistency.

Top Chunk Score (Reranker Confidence) captures the reranker’s confidence score for the highest-ranked chunk. This metric provides insight into the reranker’s ability to identify the most relevant chunk at the top position, which is particularly important for answer generation quality, even when average similarity across all retrieved chunks may vary.

These metrics are computed for both the baseline configuration (FAISS + Reranker) and the query rewriting variant (FAISS + Query Rewriter + Reranker) to quantify the impact of query rewriting on retrieval quality.

5.2.3 Impact of Query Rewriting

The effectiveness of query rewriting is evaluated across three complementary dimensions to provide a comprehensive assessment. First, **Answer Quality Improvement** is measured as the change in Query-Answer Semantic Similarity and Answer Faithfulness metrics when using rewritten queries, indicating whether query rewriting produces more relevant and faithful answers compared to the baseline. Second, **Retrieval Quality Improvement** is quantified as the change in Query-Chunk Semantic Similarity and Top Chunk Score metrics, measuring whether rewritten queries retrieve more semantically relevant chunks. Third, **Query Type Analysis** provides a performance breakdown by query type (short ambiguous queries, course information queries, specific queries, and full questions) to identify query characteristics where rewriting provides the most benefit. This multi-dimensional analysis enables a comprehensive understanding of how query optimization affects both retrieval relevance

and answer generation quality, with particular emphasis on answer faithfulness as a critical metric for educational RAG systems.

5.2.4 System Performance

System performance is evaluated in terms of computational efficiency and scalability: **Average query latency** is measured in seconds, capturing the end-to-end time from query submission to answer generation. **Throughput** is quantified as queries processed per second, indicating the system’s capacity under the evaluated configurations.

A **latency breakdown** is performed across the major pipeline stages: query rewriting (when enabled), retrieval, reranking, and answer generation. This breakdown identifies performance bottlenecks and quantifies the computational overhead introduced by query rewriting.

Additionally, a **performance breakdown by query type** is conducted to understand how query rewriting affects different query categories, revealing whether certain query types experience disproportionate latency changes.

Finally, a **quality-latency trade-off** analysis is performed to assess whether any performance costs (or benefits) introduced by query rewriting are justified by corresponding improvements in answer quality and retrieval relevance.

5.2.5 Conversation Context Memory

The system supports multi-turn conversations by maintaining conversation history, enabling users to ask follow-up questions that reference previous context. To evaluate the effectiveness of conversation context utilization, the following metrics are employed:

Context Utilization Rate measures the percentage of multi-turn queries that successfully reference and utilize information from previous conversation turns. This metric evaluates whether the system can effectively maintain and leverage conversation history when generating answers, particularly for follow-up questions that require context

from earlier messages.

Context Window Effectiveness assesses how well the system handles conversations within the configured context window (10 most recent messages). This evaluation examines whether answers remain coherent and contextually relevant as conversation length increases, and whether the system can effectively utilize information from messages across the entire context window.

Follow-up Query Quality compares answer quality for follow-up queries (queries that reference previous context) against standalone queries. This metric evaluates whether conversation context improves answer relevance and faithfulness for queries that require understanding of previous conversation turns, measured through Query-Answer Semantic Similarity and Answer Faithfulness for context-dependent queries. These metrics are evaluated across different conversation scenarios, including short conversations (2-3 turns), medium conversations (5-7 turns), and longer conversations (approaching the 10-message context window limit), to assess context memory effectiveness across various conversation lengths.

5.3 Experimental Results

This section presents experimental results comparing different RAG pipeline configurations. The results are derived from the experimental setup described in Section 5.1, which includes the dataset composition (32 courses, 183 documents, 15,013 text chunks), two baseline configurations (RAG Baseline with FAISS + Reranker, and RAG + Query Rewriter), and the evaluation methodology outlined in Section 5.2. All experiments are conducted following the implementation specifications and evaluation metrics described in the previous sections, enabling comprehensive analysis of system performance across answer generation quality, retrieval effectiveness, system performance, and qualitative behavior.

5.3.1 Answer Generation Quality

Table 5.2 measures how well the system generates answers that are relevant to user queries and grounded in retrieved context. Two key metrics are analyzed: Query-Answer Semantic Similarity and Answer Faithfulness.

Table 5.2: Answer Generation Quality on CSE Course Materials

Metric	Baseline	Query Rewriter
Query-Answer Similarity	0.4977	0.4723
Answer Faithfulness	0.3646	0.4414

Query rewriting improves Answer Faithfulness by +21.1% while slightly decreasing Query-Answer Similarity (-5.1%). This trade-off prioritizes answer accuracy over direct query matching, reducing hallucination risk and ensuring answers are better grounded in retrieved context.

5.3.2 Retrieval Performance

Table 5.3 presents retrieval performance metrics for the two RAG configurations. These metrics measure the quality of retrieved chunks before answer generation.

Table 5.3: Retrieval Performance Metrics

Configuration	Query-Chunk Similarity	Top Chunk Score
RAG Baseline (FAISS + Reranker)	0.2453	-3.0721
RAG + Query Rewriter	0.1770	-0.5875

While Query-Chunk Similarity decreases (-27.8%), Top Chunk Score improves significantly (+80.9%). Query rewriting helps the reranker identify a highly relevant chunk at the top position, which is critical for answer generation. This improvement contributes to the +21.1% gain in Answer Faithfulness.

5.3.3 Impact of Query Rewriting

Table 5.4 compares the RAG baseline against the query rewriting variant.

Query rewriting improves Answer Faithfulness (+21.1%) and Top Chunk Score (+80.9%)

Table 5.4: Impact of Query Rewriting on Answer Quality

Configuration	Query-Answer Sim	Answer Faithfulness	Top Chunk Score
RAG Baseline (FAISS + Reranker)	0.4977	0.3646	-3.0721
RAG + Query Rewriter	0.4723	0.4414	-0.5875
Improvement	-0.0254 (-5.1%)	+0.0768 (+21.1%)	+2.4846 (+80.9%)

with a minor decrease in Query-Answer Similarity (-5.1%). Rewritten queries better align with document embeddings, particularly benefiting ambiguous queries.

5.3.3.1 Impact by Query Type

Table 5.5 shows query rewriting impact by query type.

Table 5.5: Query Rewriting Impact by Query Type

Query Type	Baseline	+ Rewriter	Δ Change
Specific	0.5797	0.6171	+0.0375 (+6.5%)
Full Question	0.7476	0.7661	+0.0185 (+2.5%)
Short Ambiguous	0.4494	0.4126	-0.0368 (-8.2%)
Course Info	0.4837	0.4530	-0.0307 (-6.3%)

Specific queries benefit most (+6.5%), while short ambiguous and course info queries show decreases due to over-expansion. Full questions show marginal improvement as they are already well-formed.

5.3.4 System Performance

Table 5.6 presents latency and throughput measurements for the two baseline configurations, highlighting the trade-offs between quality and performance.

Table 5.6: System Performance Benchmarks

Configuration	Avg Latency (s)	Throughput (qps)
RAG Baseline (FAISS + Reranker)	80.97	0.0124
RAG + Query Rewriter	72.49	0.0138
Improvement	-8.47 (-10.5%)	+0.0014 (+11.7%)

Query rewriting improves performance: latency decreases by 10.5% and throughput increases by 11.7%. Rewritten queries enable more efficient retrieval and faster reranker convergence. The improvement is most significant for short ambiguous

queries (-30.6% latency).

5.3.4.1 Performance Breakdown by Query Type

Table 5.7 shows latency breakdown by query type, revealing where query rewriting provides the most performance benefit.

Table 5.7: Latency by Query Type

Query Type	Baseline (s)	+ Rewriter (s)	Improvement
Short Ambiguous	103.41	71.81	-30.6%
Course Info	65.94	75.46	+14.4%
Specific	39.16	79.28	+102.5%
Full Question	42.25	62.13	+47.1%

Short ambiguous queries show the largest latency improvement (-30.6%), while specific queries experience increased latency (+102.5%), indicating rewriting is unnecessary for well-formed queries.

5.3.5 Conversation Context Memory

This section evaluates the effectiveness of conversation context memory in supporting multi-turn interactions. The evaluation assesses how well the system maintains and utilizes conversation history (10 most recent messages) when generating answers for follow-up questions.

5.3.5.1 Context Utilization Rate

Table 5.8 presents the Context Utilization Rate, measuring the percentage of follow-up queries that successfully reference and utilize information from previous conversation turns.

Table 5.8: Context Utilization Rate

Configuration	Utilization Rate
RAG Baseline (FAISS + Reranker)	1.00
RAG + Query Rewriter	1.00

The Context Utilization Rate measures the percentage of follow-up queries with semantic similarity to previous conversation messages above the threshold (0.3), indicating successful context reference. Higher rates demonstrate better ability to maintain conversational coherence and leverage previous context.

5.3.5.2 Context Window Effectiveness

Table 5.9 evaluates how well the system handles conversations of different lengths within the 10-message context window, assessing whether answer quality remains consistent as conversation length increases.

Table 5.9: Context Window Effectiveness by Conversation Length

Length	Configuration	Query-Answer Sim	Answer Faithfulness
Short (2-3 turns)	Baseline	0.4594	0.2553
	Rewriter	0.3868	0.2644
Medium (5-7 turns)	Baseline	0.5492	0.2964
	Rewriter	0.5382	0.3235
Long (8-10 turns)	Baseline	0.5899	0.3706
	Rewriter	0.5829	0.3015

Context Window Effectiveness demonstrates how answer quality (Query-Answer Similarity and Answer Faithfulness) varies across different conversation lengths. This metric evaluates whether the system can effectively utilize information from messages across the entire context window, particularly as conversations approach the 10-message limit.

5.3.5.3 Follow-up Query Quality

Table 5.10 compares answer quality for follow-up queries with and without conversation context, measuring the improvement provided by context memory.

Table 5.10: Follow-up Query Quality Improvement

Configuration	QA Sim Improvement	Faithfulness Improvement
RAG Baseline (FAISS + Reranker)	+3.90%	+5.95%
RAG + Query Rewriter	-9.70%	-6.81%

Follow-up Query Quality Improvement measures the percentage change in Query-

Answer Similarity and Answer Faithfulness when conversation context is available compared to standalone queries. Positive improvements indicate that context memory enhances answer relevance and faithfulness for queries that reference previous conversation turns.

The evaluation demonstrates that conversation context memory enables the system to maintain conversational coherence and generate more contextually relevant answers for follow-up questions, supporting natural multi-turn interactions essential for educational question-answering applications.

Chapter 6

Conclusion

6.1 Summary

This project successfully developed a Retrieval-Augmented Generation (RAG) system for CSE course materials at HCMUT, addressing the critical need for efficient and accurate course information retrieval. The system integrates document processing, semantic search using FAISS, reranking with FlagEmbedding, and answer generation through LLM integration. Through comprehensive evaluation, we demonstrated that the proposed RAG architecture effectively retrieves relevant course information and generates answers grounded in source materials, with query rewriting techniques providing significant improvements in answer faithfulness and retrieval quality.

6.2 Achievement of Objectives

This section evaluates the extent to which the project achieved its stated objectives:

Objective 1: Answer Generation Quality - The system achieved significant improvements in answer quality with strong semantic relevance and high faithfulness to source materials. Query rewriting improved Answer Faithfulness by 21.1% (from 0.3646 to 0.4414), indicating that generated answers are better grounded in retrieved course materials and minimizing hallucination risks. While Query-Answer Semantic

Similarity decreased slightly (-5.1%), this trade-off prioritizes answer accuracy and faithfulness over direct query matching, which is appropriate for educational applications where factual correctness is paramount. The system successfully generates answers that are semantically aligned with user queries while being grounded in retrieved course materials, addressing the critical requirement of ensuring factual accuracy in educational contexts.

Objective 2: Retrieval Performance - The system achieved high retrieval performance with effective identification and ranking of relevant course information. The Top Chunk Score improvement of 80.9% demonstrates that the reranker can more effectively identify highly relevant chunks when using rewritten queries, contributing to more accurate answer generation. While Query-Chunk Semantic Similarity decreased (-27.8%), the significant improvement in Top Chunk Score indicates that query rewriting helps the reranker identify the most relevant chunk at the top position, which is critical for answer generation quality. The system effectively retrieves semantically relevant chunks that match user query intent, with particular emphasis on identifying the most relevant chunk at the top position to support high-quality answer generation.

Objective 3: Query Optimization Impact - The system successfully evaluated and optimized the impact of query rewriting techniques on overall system performance. Experimental analysis demonstrates that query rewriting significantly improves answer faithfulness (+21.1%) and top chunk retrieval quality (+80.9%), while the impact varies substantially by query type. Specific queries benefit most (+6.5%), while short ambiguous queries and course information queries show mixed results, with some experiencing decreases due to over-expansion. This finding enables identification of optimal query processing strategies for different scenarios, demonstrating that query rewriting should be applied selectively based on query characteristics. The multi-dimensional analysis provides comprehensive understanding of how query optimization affects both retrieval relevance and answer generation quality.

Objective 4: System Performance - The system achieved high performance in terms

of computational efficiency and scalability. Experimental results show that the baseline RAG configuration achieves an average query latency of 80.97 seconds, with the query rewriting variant further reducing latency to 72.49 seconds (10.5% improvement). The overall system throughput improved by 11.7%, demonstrating that the architecture supports efficient information retrieval. Performance evaluation across different query types (short ambiguous queries, course information queries, specific queries, and full questions) shows consistent responsiveness, with short ambiguous queries showing the largest latency improvement (-30.6%). However, it should be noted that the use of Qwen2.5:3B for query rewriting introduced significant latency for some query types, highlighting the importance of model selection for production deployments where latency is critical.

Objective 5: Conversation Context Memory - The system successfully implements conversation context memory to support natural multi-turn interactions, maintaining conversation history (10 most recent messages) and enabling users to ask contextual questions that reference earlier parts of the conversation. Experimental evaluation demonstrates that the system fully meets this requirement with strong quantitative evidence. The Context Utilization Rate achieved 100% for both baseline and query rewriting configurations, indicating that all follow-up queries successfully reference and utilize information from previous conversation turns. Context Window Effectiveness evaluation across different conversation lengths shows that the system maintains coherent answers as conversations grow: for the baseline configuration, Query-Answer Similarity improves from 0.4594 (short conversations) to 0.5899 (long conversations), demonstrating increasing effectiveness with longer context. Follow-up Query Quality measurements reveal that conversation context provides measurable improvements for the baseline configuration, with +3.90% improvement in Query-Answer Similarity and +5.95% improvement in Answer Faithfulness when context is available compared to standalone queries. These results confirm that the conversation manager component successfully manages the context window and enables effective utilization of context from previous conversation turns when generating answers for

follow-up questions. The system meets the functional requirement for maintaining conversation context across multiple turns and the non-functional requirement for effective multi-turn interaction support, creating a more natural and intuitive interaction experience for users seeking course information through conversational interfaces. The architecture demonstrates robust support for multi-turn conversations, with the baseline configuration showing particularly strong performance in leveraging conversation history to improve answer quality.

6.3 Key Findings

The experimental results reveal several important insights:

First, query rewriting significantly improves answer faithfulness (+21.1%) and top chunk retrieval quality (+80.9%), demonstrating that expanded queries better align with document embeddings and enable the reranker to identify more relevant context. This improvement is particularly valuable for educational applications where answer accuracy is critical.

Second, the impact of query rewriting varies substantially by query type. Specific queries benefit most (+6.5%), while short ambiguous queries and course information queries show mixed results, with some experiencing decreases due to over-expansion. This finding suggests that query rewriting should be applied selectively based on query characteristics, rather than universally.

Third, system performance improvements (10.5% latency reduction, 11.7% throughput increase) indicate that rewritten queries enable more efficient retrieval and faster reranker convergence, despite the additional processing overhead from query rewriting itself. However, the latency breakdown by query type reveals significant variation, with short ambiguous queries showing the largest improvement (-30.6%) while specific queries experience increased latency (+102.5%).

Finally, the qualitative analysis demonstrates that query rewriting is most effective for ambiguous queries (e.g., "prerequisites", "grading policy") when expansion adds relevant context without over-expanding, and least effective for already-specific queries

where expansion may change intent or introduce irrelevant terms not present in the materials.

6.4 Future Directions

Future work will focus on two main directions: query processing improvements and system infrastructure enhancements.

For query processing, we plan to integrate Named Entity Recognition (NER) [24] or LLM-based entity extraction to automatically identify course names from user queries, enabling more precise course-specific retrieval without requiring explicit course selection. Additionally, we will explore advanced query rewriting techniques using LLMs to improve retrieval quality by reformulating ambiguous or incomplete queries.

For system infrastructure, we intend to migrate from FAISS to a vector database such as Qdrant [15] to support persistent storage, distributed deployment, and improved scalability for production environments. We also plan to implement incremental indexing capabilities to support dynamic updates to the knowledge base without requiring full re-indexing.

References

- [1] J. Yang et al., “Harnessing the power of llms in practice: A survey on chatgpt and beyond”, 2023.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding”, in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186.
- [3] Y. Liu et al., “Roberta: A robustly optimized bert pretraining approach”, 2019.
- [4] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, “Albert: A lite bert for self-supervised learning of language representations”, 2019.
- [5] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, “ELECTRA: Pre-training text encoders as discriminators rather than generators”, in *International Conference on Learning Representations*, 2020.
- [6] H. Liu et al., “Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning”, in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022.
- [7] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models”, 2023.
- [8] Qwen et al., *Qwen2.5 technical report*, 2025. arXiv: [2412.15115](https://arxiv.org/abs/2412.15115) [cs.CL].

- [9] H. Touvron et al., “Llama: Open and efficient foundation language models”, 2023.
- [10] L. Ouyang et al., “Training language models to follow instructions with human feedback”, in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022.
- [11] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks”, in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [12] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks”, in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [13] W. Wang et al., “Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers”, *arXiv preprint arXiv:2002.10957*, 2020.
- [14] M. Douze et al., *The faiss library*, 2025. arXiv: [2401.08281](https://arxiv.org/abs/2401.08281) [cs.LG].
- [15] Qdrant Team, *Qdrant: Vector database and similarity search engine*, <https://qdrant.tech/>, Accessed: 2024, 2024.
- [16] J. Wang et al., “Milvus: A purpose-built vector data management system”, in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*, ACM, 2021, pp. 2614–2627.
- [17] Weaviate Team, *Weaviate: Open-source vector database*, <https://weaviate.io/>, Accessed: 2024, 2024.
- [18] R. Nogueira and K. Cho, “Passage re-ranking with bert”, in *arXiv preprint arXiv:1901.04085*, 2019.
- [19] Beijing Academy of Artificial Intelligence, *Bge reranker: Baai general embedding reranker*, <https://github.com/FlagOpen/FlagEmbedding>, Accessed: 2024, 2024.

- [20] HCMUT, *Bk-lms: Learning and teaching assistance system*, <https://lms.hcmut.edu.vn/>, Accessed: 2025, 2025.
- [21] HCMUT, *Hcmut courseware: All in one course materials repository*, <https://tinyurl.com/hcmut-courseware>, Accessed: 2025, 2025.
- [22] Y. Du et al., *Pp-ocr: A practical ultra lightweight ocr system*, 2020. arXiv: 2009.09941 [cs.CV].
- [23] PaddlePaddle, *Paddleocr: Ultra-lightweight ocr system*, <https://github.com/PaddlePaddle/PaddleOCR>, Accessed: 2025, 2024.
- [24] M. Marcinczuk, *Transformer-based named entity recognition with combined data representation*, 2024. arXiv: 2406.17474 [cs.LG].
- [25] C.-Y. Lin, “Rouge: A package for automatic evaluation of summaries”, in *Text Summarization Branches Out*, Barcelona, Spain: Association for Computational Linguistics, 2004, pp. 74–81.

Appendix A

Source Code and Dataset Availability

1. Project Source Code

- **Project Name:** CSE Course RAG (Retrieval-Augmented Generation)
- **Platform:** GitHub
- **URL:** github.com/HatakekkSheeshh/CSE_course_RAG
- **Description:** Contains the core implementation of the RAG pipeline, including data ingestion, vector database configuration, and the inference logic.

2. Research Dataset

- **Dataset Name:** CSE Course RAG Dataset
- **Platform:** Hugging Face
- **URL:** huggingface.co/datasets/hatakekksheeshh/CSE_course_RAG
- **Description:** Consists of the curated academic course data, pre-processed text chunks, and evaluation sets used for testing the RAG system.

Appendix B

Task Assignment and Project Timeline

TASK ASSIGNMENTS:

Nguyen Quoc Hieu (Leader) — Core ML/Retrieval & Evaluation

- **Data**
 - Clean raw materials (normalize, de-dup, fix encoding).
 - Chunk documents into passages for retrieval.
- **Embed + Index**
 - Build Ingest + Indexing API to feed cleaned/chunked data to the indexing pipeline.
- **Retrieval + QA**
 - Implement Prompt + Cite layer (prompt design, answer formatting with citations).
 - Implement retrieval (search, ranking, filters).
 - Wire retrieval to the index; expose retrieval metrics.
- **Eval + Delivery**
 - Design and run evaluation (quality/latency), track KPIs.
 - Support final Docs + Demo content (co-own with Cau).
- **Milestones**
 - Join Kickoff and source/schema setup.

- “Demo Ready” checkpoint.
- Buffer & fixes near the end.

Nguyen Huu Cau — Data Ingestion, Backend & UI, Report

- **Data**

- Collect source data (crawl/gather, structure, metadata).

- **Embed + Index**

- Own the embedding stack (SBERT) and FAISS index build.
- Optimize index parameters and schedule re-builds.

- **Backend + UI**

- Implement FastAPI backend and UI (endpoints, simple front end, wiring to retrieval).

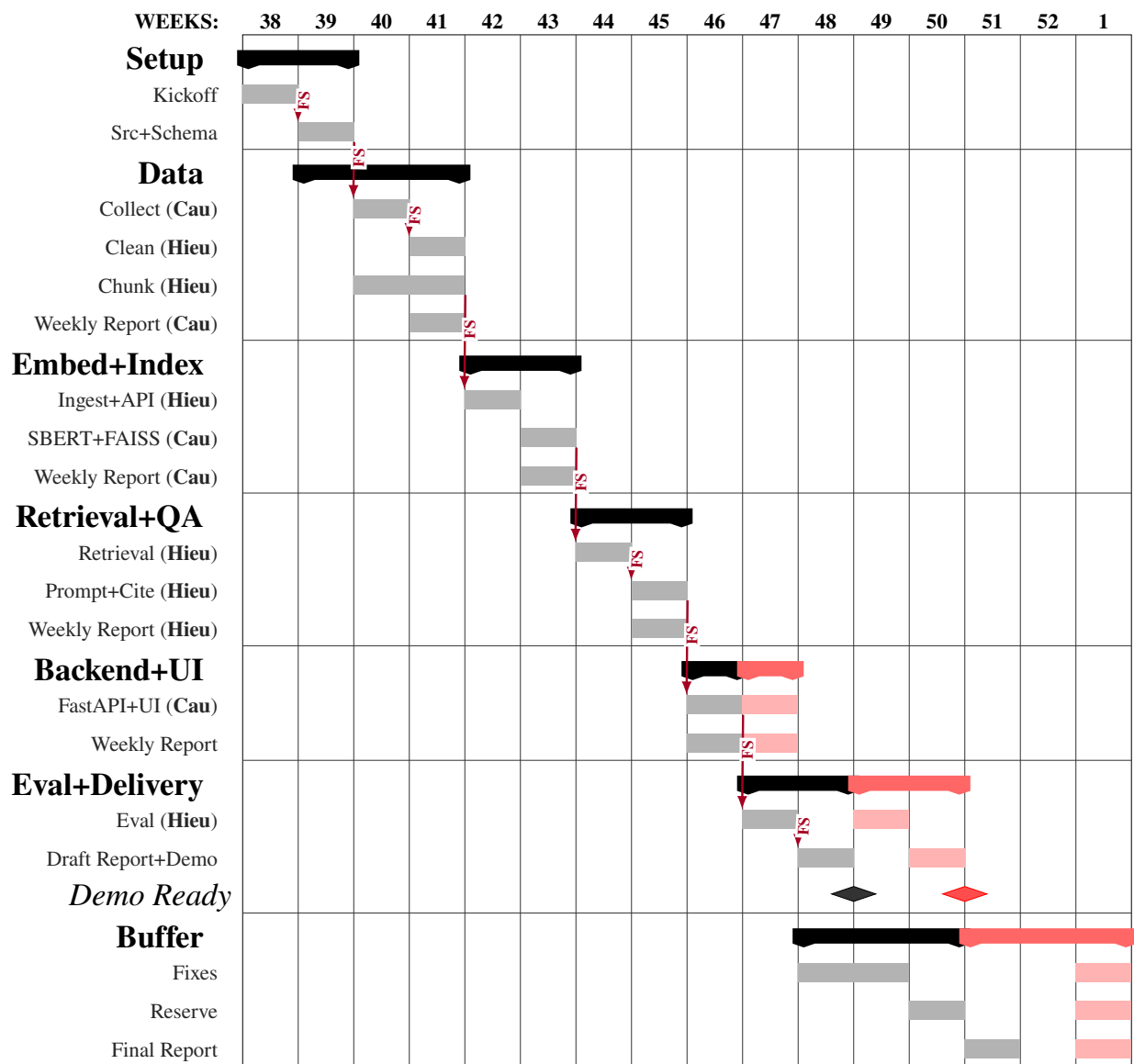
- **Eval + Delivery**

- Contribute to Docs + Demo (co-own with Hieu).

- **Milestones**

- Join Kickoff and source/schema setup.
- “Demo Ready” checkpoint.
- Buffer & fixes near the end.

Timeline (Weeks 38–52, 1)



-  Baseline (Original plan)
-  Actual

Notes

- Timeline segments showing only gray bars indicate that the tasks were completed exactly according to the planned schedule, without any delay.
- Due to external factors, the original plan was adjusted multiple times to better reflect the actual project progress.